设计赛道文档

全国大学生计算机系统能力大赛——操作系统设计赛

功能挑战赛道

队伍名称：**Go_Go_Go**

学校：**华南师范大学**

成员：**张乐桁、陈韵、潘利源**

赛题：**面向 Rust OS 的高性能可信存储实现**

2026 年 8 月 17 日

全部代码、证据、文档和演示材料已按大赛要求提交至指定托管平台。

目录

1 目标描述 8
1.1 项目背景与意义 9
1.1.1 可信存储面临的挑战 9
1.1.2 为什么选择星绽操作系统（Asterinas） 9
1.1.3 项目定位 10
1.2 项目目标 10
1.2.1 功能目标一：dm-verity 只读完整性校验 10
1.2.2 功能目标二：dm-crypt 透明加密 10
1.2.3 功能目标三：受保护根文件系统 11
1.2.4 功能目标四：Linux 兼容性与工具链支持 11
1.2.5 功能目标五：性能与启动要求 11
1.2.6 技术探索目标（创新性） 11
1.3 赛题要求映射 12
1.4 双目标架构创新 12
1.5 测试环境与验证方法 13
1.6 预期成果 13
1.7 小结 14
2 比赛题目分析和相关资料调研 15
2.1 比赛题目分析 15
2.1.1 赛题背景与核心要求 15
2.1.2 赛题特征解析 16
2.1.3 为什么这个赛题具有挑战性 16
2.1.4 赛题与操作系统课程知识的关联 17
2.2 相关技术调研 17
2.2.1 Linux Device Mapper 框架深度分析 17
2.2.2 星绽操作系统（Asterinas）架构深入分析 18
2.2.3 密码学标准与算法调研 18
2.2.4 Rust 在可信存储领域的独特优势 19
2.2.5 参考实现与开源生态调研 20
2.2.6 兼容性验证策略调研 20

2.3 技术可行性评估 . . . 21
2.4 小结 . . . 21

3 系统框架设计 . . . 23
3.1 设计原则 . . . 23
3.2 核心抽象：BlockDevice Trait . . . 24
3.3 可组合的映射器链模式 . . . 24
3.4 核心模块分解 . . . 25
3.5 双目标架构：跨环境可移植性 . . . 26
3.6 数据流架构 . . . 26
3.6.1 dm-crypt 数据流 . . . 26
3.6.2 dm-verity 数据流 . . . 27
3.7 线程安全保证 . . . 27
3.8 性能可观测性设计 . . . 27
3.9 运行时控制面（Linux 兼容） . . . 28
3.10 安全能力传播 . . . 28
3.11 Asterinas 内核集成架构 . . . 29
3.12 小结 . . . 29

4 开发计划 . . . 30
4.1 开发方法论 . . . 30
4.2 阶段划分与时间安排 . . . 31
4.3 第一阶段：用户态核心实现 . . . 31
4.3.1 目标 . . . 31
4.3.2 任务分解 . . . 31
4.3.3 交付物与验收标准 . . . 32
4.3.4 实际完成情况 . . . 32
4.4 第二阶段：Linux 兼容性与工具链 . . . 32
4.4.1 目标 . . . 32
4.4.2 任务分解 . . . 32
4.4.3 交付物与验收标准 . . . 32
4.4.4 实际完成情况 . . . 33
4.5 第三阶段：内核补丁与 BIO 集成 . . . 33
4.5.1 目标 . . . 33
4.5.2 任务分解 . . . 33
4.5.3 交付物与验收标准 . . . 33
4.5.4 实际完成情况 . . . 33
4.6 第四阶段：性能优化与基准框架 . . . 34
4.6.1 目标 . . . 34

4.6.2 任务分解 . . . 34
4.6.3 交付物与验收标准 . . . 34
4.6.4 实际完成情况 . . . 34
4.7 第五阶段：组合栈与最终验证 . . . 34
4.7.1 目标 . . . 34
4.7.2 任务分解 . . . 35
4.7.3 交付物与验收标准 . . . 35
4.7.4 实际完成情况 . . . 35
4.8 第六阶段：文档整理与答辩准备 . . . 35
4.8.1 目标 . . . 35
4.8.2 任务分解 . . . 35
4.8.3 交付物与验收标准 . . . 36
4.8.4 实际完成情况 . . . 36
4.9 里程碑与关键交付物汇总 . . . 37
4.10 开发环境与工具链 . . . 38
4.11 风险应对 . . . 39
4.12 计划执行偏差分析 . . . 39
4.13 小结 . . . 40

5 比赛过程中的重要进展 . . . 41
5.1 概述 . . . 41
5.2 第一阶段：用户态核心实现（2026-06-23 至 2026-06-28） . . . 42
5.2.1 6 月 23 日：初始提交与核心框架搭建 . . . 42
5.2.2 6 月 23 日–28 日：用户态工具与文档完善 . . . 42
5.3 第二阶段：Linux 兼容性与内核集成启动（2026-06-29 至 2026-07-14） 43
5.3.1 6 月 29 日：Linux XTS 兼容性验证 . . . 43
5.3.2 7 月 14 日：Asterinas 内核补丁 0001–0004 完成 . . . 43
5.4 第三阶段：性能优化与基准框架（2026-07-28 至 2026-08-03） . . . 44
5.4.1 7 月 28 日：补丁 0005 – 热路径优化 . . . 44
5.4.2 8 月 3 日：补丁 0006 – 性能归因与正式基准合同 . . . 45
5.5 第四阶段：性能矩阵执行与证据归档（2026-08-09 至 2026-08-11） . . 45
5.5.1 8 月 9 日：真实性审计与重新执行 . . . 45
5.5.2 8 月 10 日：补充性能测试与证据整理 . . . 46
5.5.3 8 月 11 日：组合栈（补丁 0007）与最终整合 . . . 46
5.6 第五阶段：文档与答辩材料完成（2026-08-11 至 2026-08-14） . . . 47
5.7 关键里程碑时间线 . . . 48
5.8 Git 提交记录概览 . . . 48
5.9 各阶段代码量统计 . . . 50

5.10 阶段性成果总结 51
5.11 小结 51

6 系统测试情况 52
6.1 测试体系概述 53
6.2 测试环境基线 53
6.3 单元测试 54
6.4 Linux 兼容性测试 54
6.4.1 测试目的 54
6.4.2 测试方法 54
6.4.3 测试结果 54
6.5 Asterinas KVM 正确性测试 55
6.5.1 回归测试（R01–R03） 55
6.5.2 dm-crypt 加密测试（C01–C05） 55
6.5.3 dm-verity 完整性测试（V01–V07） 56
6.5.4 物理篡改清单详情 58
6.6 性能测试 58
6.6.1 FIO 矩阵 58
6.6.2 补充性能测试 60
6.7 数据真实性与审计 60
6.7.1 历史问题处理 60
6.7.2 正式数据标准 60
6.8 静态检查与门禁 61
6.9 证据可复核性 61
6.10 测试门禁汇总 63
6.11 小结 63

7 遇到的主要问题和解决方法 64
7.1 概述 64
7.2 问题解决方法论 64
7.3 块大小与扇区语义问题 65
7.3.1 问题描述 65
7.3.2 分析过程 65
7.3.3 解决方案 65
7.3.4 验证结果 66
7.4 BIO 生命周期与错误传播问题 66
7.4.1 问题描述 66
7.4.2 分析过程 66
7.4.3 解决方案 67

7.4.4 验证结果 . . . 67
7.5 dm-verity 信任边界与缓存安全 . . . 67
7.5.1 问题描述 . . . 67
7.5.2 分析过程 . . . 67
7.5.3 解决方案 . . . 68
7.5.4 验证结果 . . . 68
7.6 性能测试真实性问题 . . . 68
7.6.1 问题描述 . . . 68
7.6.2 分析过程 . . . 68
7.6.3 解决方案 . . . 69
7.6.4 验证结果 . . . 69
7.7 环境下载与持久化缓存问题 . . . 69
7.7.1 问题描述 . . . 69
7.7.2 分析过程 . . . 69
7.7.3 解决方案 . . . 70
7.7.4 验证结果 . . . 70
7.8 pivot_root 限制与引导问题 . . . 70
7.8.1 问题描述 . . . 70
7.8.2 分析过程 . . . 70
7.8.3 解决方案 . . . 70
7.8.4 验证结果 . . . 71
7.9 组合栈的命名与设备发现问题 . . . 71
7.9.1 问题描述 . . . 71
7.9.2 分析过程 . . . 71
7.9.3 解决方案 . . . 71
7.9.4 验证结果 . . . 72
7.10 内核态与用户态同步原语差异 . . . 72
7.10.1 问题描述 . . . 72
7.10.2 分析过程 . . . 72
7.10.3 解决方案 . . . 72
7.10.4 验证结果 . . . 72
7.11 AES-NI 的 FPU/XMM 状态保存问题 . . . 73
7.11.1 问题描述 . . . 73
7.11.2 分析过程 . . . 73
7.11.3 解决方案 . . . 73
7.11.4 验证结果 . . . 74
7.12 问题解决效果统计 . . . 74

7.13 小结 74

8 分工和协作 76

8.1 队员信息 76
8.2 分工详情 77
8.2.1 潘利源（核心架构师） 77
8.2.2 陈韵（jqtxzs，测试工程师） 77
8.2.3 张乐桁（lehengzhang，文档工程师） 78
8.3 协作方式 78
8.3.1 版本管理 78
8.3.2 沟通与同步 78
8.3.3 代码审查 79
8.3.4 决策机制 79
8.3.5 关键决策记录 79
8.4 AI 工具辅助协作 79
8.5 团队贡献量化 80
8.6 开发节奏与协作模式亮点 81
8.7 团队协作感悟 81
8.7.1 潘利源 81
8.7.2 陈韵 81
8.7.3 张乐桁 82
8.8 小结 82

9 提交仓库目录和文件描述 83

9.1 文件统计概览 84
9.2 仓库顶层结构 84
9.3 脚本分类说明 86
9.4 文档目录说明 88
9.5 关键证据文件说明 91
9.6 与大赛提交要求对照 92
9.7 构建与测试产物（不提交） 93
9.8 小结 93

10 比赛收获 94

10.1 概述 94
10.2 技术成长 94
10.2.1 Rust 系统编程的深入实践 94
10.2.2 密码学工程实践 95
10.2.3 操作系统内核开发 95

10.2.4 性能工程 96
10.3 工程能力提升 96
10.3.1 大型项目开发与协作 96
10.3.2 自动化与可复现性 97
10.3.3 文档写作与学术表达 97
10.4 团队协作收获 97
10.4.1 分工与互补 97
10.4.2 共识决策与代码审查 98
10.4.3 团队成员感想 98
10.5 对 Rust OS 生态的认识 99
10.6 关键技术指标达成总结 99
10.7 未来展望详细路线图 100
10.8 个人感悟 100
10.8.1 潘利源 100
10.8.2 陈韵 100
10.8.3 张乐桁 100
10.9 感谢 101
10.10小结 101

第 1 章　目标描述

本章执行摘要

本项目在星绽操作系统（Asterinas）上使用 Rust 语言实现了与 Linux 兼容的 `dm-crypt` 透明加密和 `dm-verity` 完整性校验功能，填补了星绽生态在可信存储领域的关键空白。核心算法（AES-256-XTS、SHA-256 Merkle Tree）与 Linux 内核逐字节兼容，并通过统一的 `BlockDevice` trait 抽象，同时服务于用户态（`dmctl` 工具）和内核态（`aster-dm` 组件）。项目已通过 KVM 正确性测试（R01–R03、C01–C05、V01–V07 全部 PASS）和五轮性能基准（46 个有效 FIO 单元格），并实现了 AES-NI 硬件加速、认证后哈希缓存、有界临时缓冲池等性能优化措施。全部代码、证据、文档和演示材料已按赛题要求提交至指定托管平台。

表 1.1: 赛题指标完成情况速查

赛题指标	**状态**	**证据**
dm-verity 基础功能(哈希树、SHA-256、启动参数)	已完成	R03/V01 PASS
dm-crypt 基础功能（透明加解密、cryptsetup 支撑）	已完成	R01/C01–C05 PASS
根文件系统加密	已完成	R02 PASS
性能与启动要求	已完成	五轮性能矩阵 PASS
技术探索（Rust OS 存储优化）	已完成	AES-NI、hash cache、scratch pool

赛题评审要点对应速查

评审要点	权重	本项目完成情况
实现完整度	50%	全部五项技术指标完成，R01–R03、C01–C05、V01–V07 全部 PASS
文档完整性	20%	提供技术设计文档（9 章 + 附录）、设计赛道文档（10 章）、用户手册、AI 声明
Demo 质量	20%	提供一键演示脚本（`final_demo.sh`）和演示视频，含 KVM 根切换实况
创新性	10%	双目标架构、认证后缓存、AES-NI 集成、BIO 复用、阶段归因统计

1.1 项目背景与意义

1.1.1 可信存储面临的挑战

在机密计算和云计算场景中，持久化存储介质（如根文件系统镜像、用户数据卷）通常位于不可信的主机环境中，面临三类核心安全威胁：

1. **机密性威胁**：物理介质被窃取或镜像被直接读取时，明文数据泄露；
2. **完整性威胁**：系统镜像被离线篡改（植入后门、替换关键二进制文件），运行时难以察觉；
3. **替换威胁**：整个镜像被替换为攻击者构造的恶意版本，系统在启动时即被攻陷。

Linux 内核通过 Device Mapper 框架提供的 `dm-crypt`（透明加密）和 `dm-verity`（只读完整性校验）机制，已广泛应用于全盘加密、安全启动和容器镜像保护，是业界公认的成熟方案。

1.1.2 为什么选择星绽操作系统（Asterinas）

星绽操作系统是一个采用创新性 Framekernel 架构的 Rust 操作系统，其核心特点包括：

- **内存安全**：基于 Rust 的所有权与生命周期机制，从根源上规避了悬垂指针、use-after-free 和缓冲区溢出等内存安全问题；
- **TCB 最小化**：通过 OSTD 框架将内核核心组件运行于非特权级，缩小了可信计算基；
- **机密计算场景**：星绽计划作为 Guest Kernel 运行于可信虚拟机中，持久化存储的安全保护是其生态的关键一环。

然而，星绽的内核接口与 Linux 差异较大（BIO 系统、设备注册、启动流程均不同），无法直接复用 Linux 的 Device Mapper 实现，需要从头设计 Rust 实现并适配星绽内核。

1.1.3 项目定位

本项目旨在为星绽操作系统填补这一空白，使用 Rust 语言自主实现一套与 Linux 兼容、可组合、可验证的块设备安全映射层。这不仅是对星绽存储安全能力的关键补充，也是对 Rust 在操作系统内核中实现密码学与块设备映射技术的一次深入探索。项目的成功验证了 Rust 在系统软件安全存储领域的可行性与优势。

1.2 项目目标

本项目的核心目标是在星绽操作系统上实现一套可组合、可验证的块设备安全映射层，具体分解为以下五大技术指标，全面覆盖赛题要求：

1.2.1 功能目标一：dm-verity 只读完整性校验

- 基于 SHA-256 哈希算法构建 Merkle 哈希树（块大小 4096 字节）；
- 支持 on-disk 哈希树布局，根哈希作为可信锚点通过启动参数（`rootfs_verity.hash`）注入；
- 读路径逐层验证数据块至根哈希，任意篡改（数据块或中间哈希块）均会被检测并拒绝；
- 强制执行只读语义，禁止写入操作；
- 支持可配置的认证后哈希缓存（默认 256 块），缓存遵循“先认证、后插入”原则，不削弱安全边界。

1.2.2 功能目标二：dm-crypt 透明加密

- 支持 AES-256-XTS 加密模式（NIST SP 800-38E 标准），密钥长度 512 位；
- 采用 `plain64` IV 生成策略（64 位扇区号小端序），与 Linux 内核 `aes-xts-plain64` 完全兼容——已通过固定向量逐字节比对验证（SHA-256: `4334c17f...`）；
- 支持 Linux `dm-crypt` 表格式解析（包括 `iv_offset`、`data_offset`、`allow_discards`、`no_read_workqueue` 等标准选项），不支持的选项会显式报错；
- 提供真实的块设备读写路径，确保写入底层介质的为密文，读取时自动解密为明文；
- 支持有界临时缓冲池（scratch pool）减少热路径动态分配，默认容量 32。

1.2.3 功能目标三：受保护根文件系统

- 支持通过内核启动参数配置 dm-crypt 加密根文件系统(dm_crypt.device、dm_crypt.key);
- 支持通过启动参数配置 dm-verity 校验根文件系统(rootfs_verity.scheme、rootfs_verity.ha
- 支持组合栈（dm_stack=crypt,verity），同时提供机密性和完整性保护;
- 在 initramfs 中完成 mapper 挂载，并通过 pivot_root 切换到受保护根，启动日志输出 PASS system root switched to protected mapper。

1.2.4 功能目标四：Linux 兼容性与工具链支持

- **控制面兼容**:实现 /dev/mapper/control 控制节点,支持 dm_ioctl 子集(DM_DEV_CREATE、DM_TABLE_LOAD、DM_DEV_SUSPEND、DM_DEV_STATUS、DM_DEV_REMOVE），使标准 cryptsetup（plain 模式）能够创建、读取和删除加密映射器;
- **职责分层**: LUKS2 的 PBKDF、keyslot 与 metadata 继续由用户态 cryptsetup 负责，内核只消费其下发的 dm-crypt table——这与 Linux 的职责分层一致;
- **用户态工具**: 提供 dmctl 命令行工具，支持镜像初始化、加密读写、verity 格式化、验证、演示和基准测试，便于快速验证和调试。

1.2.5 功能目标五：性能与启动要求

赛题要求“在 I/O 吞吐及延迟上接近 Linux，且 dm-verity 不显著增加 boot 时间”。本项目在 Linux QEMU virtio 和 Asterinas KVM 两个平台上分别完成五轮基准测试，结果如下:

- **Asterinas 256 MiB 正式合同 4K 顺序读**: raw 4.249 MiB/s，dm-crypt 4.059 MiB/s（相对配对 raw 95.6%），dm-verity 3.815 MiB/s（相对配对 raw 87.3%）;
- **Asterinas KVM 13 格 mapper 4K 顺序读中位带宽**: dm-crypt 8.908 MiB/s，dm-verity 7.505 MiB/s（独立矩阵，不与 4.249 计算保留率）;
- **Linux QEMU virtio 4K 顺序读中位带宽**: raw 11.391 MiB/s，dm-crypt 10.942 MiB/s，dm-verity 10.704 MiB/s（QEMU/KVM、8 GiB、libaio，作为对照参考，不跨平台计算倍率）;
- **启动开销**: dm-crypt 相对 plain 根中位开销 0.58%，dm-verity 相对 plain 根中位开销 2.75%（均为五轮中位数），满足“不显著增加”的要求;
- 提供可复现的 FIO（46 个有效 PASS 单元格）、Filebench（5+5 轮）、SQLite（5+5+5 轮）和启动耗时（5+5+5 轮）矩阵，每个单元格至少五轮，报告中位数和标准差。

1.2.6 技术探索目标（创新性）

- **AES-NI 硬件加速**: 在内核中集成 AES-NI 指令，在保存/恢复 FPU/XMM 状态的前提下执行加密/解密，启动日志明确显示 backend=AES-NI;

- **有界临时缓冲池**：实现 `CryptScratchPool`，默认容量 32，支持 A/B 对照验证（设为 0 可关闭），量化减少动态分配带来的延迟；
- **认证后哈希缓存**：只有沿完整 Merkle 路径验证成功（到达 root hash）的哈希块才允许插入缓存，缓存命中时直接使用已认证的哈希块（计数器 `auth_shortcuts` 记录），在不削弱安全边界的前提下减少重复 SHA-256 计算；
- **BIO 数据复用**：对 4 KiB 对齐的 BIO 直接复用已完成的数据块进行校验，避免因校验而重复读取同一数据块（计数器 `data_rereads=0` 验证该优化生效）；
- **可组合安全能力传播**：设计 `DmStackBuilder`（用户态）和 `dm_stack` 内核启动参数，使安全能力（confidentiality、integrity、read_only）能够自动传播和汇总。

1.3 赛题要求映射

本项目的设计严格遵循全国操作系统设计赛功能挑战赛题的五项技术指标，完整对应如下：

赛题要求	本项目实现	验证证据
dm-verity 基础功能（哈希树、SHA-256、启动参数）	`hash_tree.rs`、`sha256.rs`、`rootfs_verity.*` 参数解析	R03/V01 PASS
dm-crypt 基础功能（透明加解密、支撑 cryptsetup）	`aes-xts-plain64` 逐字节兼容；`/dev/mapper/control` 支持 `cryptsetup`	R01/C01–C05 PASS
根文件系统加密	`dm_crypt.*` 参数创建加密 mapper，`dm-root` 模式切换根	R02 PASS
性能与启动要求	分平台五轮基准完成；plain/verity 启动开销 2.75%	46 个 FIO PASS 单元格
技术探索（Rust OS 存储优化）	AES-NI、scratch pool、认证后 hash cache、BIO 复用、阶段归因统计	`/proc/dm-*-stats` 输出

1.4 双目标架构创新

本项目的一个核心创新是**双目标架构设计**：核心逻辑被实现为一个与平台无关的 Rust 库（`dm` crate），该库同时服务于两个目标环境：

- **用户态(`dmctl`)**:用于演示、测试和基准测试,使用 `FileBlockDevice` 或 `MemoryBlockDevice` 作为后端，能够在宿主机上快速验证加密和完整性机制的正确性；

- **内核态 (aster-dm)**: 将相同的核心逻辑移植到内核空间，包装真实的 virtio-blk 设备，通过 BIO 路径处理 I/O，使星绽能够挂载受保护的文件系统。内核集成代码位于 asterinas/kernel/comps/dm/，通过 asterinas_port/patches/ 中的 7 个补丁可追溯完整演进历史。

这种设计确保了用户态与内核态的逻辑一致性，使得测试和调试更加高效，同时也为后续的性能优化和功能扩展提供了统一的抽象基础。两个环境共享相同的算法实现（AES-XTS、SHA-256、Merkle 树验证），但适配层（BIO vs 同步文件 I/O、自旋锁 vs 互斥锁、on-disk tree vs in-memory tree）完全隔离，互不影响。

1.5 测试环境与验证方法

赛题要求“基于 QEMU 虚拟机搭建标准化测试环境，存储设备统一通过 virtio-blk 标准化虚拟化接口实现管理与数据交互”。本项目严格遵循：

- **测试环境**: Asterinas 官方开发镜像 asterinas/asterinas:0.18.0-20260618，QEMU/KVM，-cpu host，1 vCPU，8 GiB 内存；
- **存储设备**：virtio-blk 设备（vdc），通过 qemu_args.sh 脚本挂载；
- **正确性验证**：通过 R01–R03、C01–C05、V01–V07 共 12 组测试用例，覆盖回归、加密、篡改三大类；
- **性能验证**：通过 FIO、Filebench、SQLite 和启动耗时四类测试，每个场景固定五轮，交替运行，保留原始数据。

1.6 预期成果

- **可交付代码**:
  - crates/dm/：平台无关的核心库（约 3000 行 Rust）；
  - crates/dmctl/：用户态命令行工具（约 800 行 Rust）；
  - asterinas_port/patches/0001--0007: Asterinas 内核集成补丁（约 4000 行内核代码）；
  - scripts/：构建、测试、性能运行脚本（约 2000 行 Shell/Python）。
- **可复核证据**（全部位于 docs/evidence/）:
  - 正确性测试 R01–R03、C01–C05、V01–V07 的原始串口日志；
  - Linux aes-xts-plain64 固定向量逐字节兼容性 SHA-256；
  - 物理篡改清单（PHYSICAL_MUTATION_MANIFEST.md）及对应的拒绝日志；
  - 最终性能矩阵（46 个 PASS 单元格）及五轮原始 JSON/CSV；
  - Filebench（5+5 轮）、SQLite（5+5+5 轮）、启动耗时（5+5+5 轮）原始记录。
- **技术文档与答辩材料**:
  - 技术设计文档（LaTeX，9 章 + 附录，含架构图、流程图）；

  - 本设计赛道文档（LaTeX，10 章，严格按大赛大纲组织）；
  - 用户手册（`README.md`，含完整命令参考和故障排查）；
  - 答辩 PPT（`presentation_project.pptx`）；
  - 演示视频（含运行环境介绍与受保护根文件系统启动）；
  - AI 工具使用声明（`ai_tool_disclosure.md`）。
- **学术/工程贡献**：
  - 为 Rust 操作系统生态贡献一套经过验证的安全存储框架；
  - 验证了 Rust 在操作系统内核中实现密码学与块设备映射的技术可行性；
  - 为星绽 OS 后续的可信启动、远程证明和密钥管理提供了基础。

1.7 小结

本章明确了项目的背景、技术目标、赛题映射、双目标架构创新、测试环境和预期交付成果。项目已按期完成全部 P0 功能目标：

- `dm-crypt`：AES-256-XTS 与 Linux 逐字节兼容，`cryptsetup` 控制面已接通；
- `dm-verity`：on-disk Merkle 树验证与根切换已通过正反例验证；
- 受保护根文件系统：加密根（R02）和校验根（R03/V01）均已通过 `pivot_root` 验证；
- 性能基准：46 个 FIO PASS 单元格、Filebench/SQLite/Boot 五轮矩阵全部完成，启动开销 $< 3\%$；
- 技术探索：AES-NI、scratch pool、认证后 hash cache、BIO 复用均已实现并量化验证。

项目成果全面覆盖赛题五项技术指标和四项评审要点，所有代码、证据、文档和演示材料均已按大赛要求提交至指定托管平台。后续章节将详细阐述系统架构设计、核心模块实现、测试验证过程及开发中的关键问题与解决方法。

第 2 章　比赛题目分析和相关资料调研

本章调研发现速览

调研方向	核心发现	对本项目的启示
Linux Device Mapper	dm-crypt 使用 XTS-AES + plain64 IV；dm-verity 使用 SHA-256 Merkle 树 + on-disk 哈希树布局	必须与 Linux 逐字节兼容，否则无法与 cryptsetup 互操作
Asterinas 内核架构	Framekernel + OSTD + BIO 异步 I/O + `no_std` 环境	无法复用 Linux 代码，需从头设计 Rust 实现，适配 BIO 路径
AES-XTS 标准	NIST SP 800-38E 定义 XTS-AES；Linux 使用 AES-256-XTS 双密钥	需严格遵循标准，并通过固定向量验证兼容性
SHA-256 标准	FIPS 180-4 定义；Linux dm-verity 使用 32 字节摘要	需实现纯 Rust SHA-256，无外部依赖
Rust 系统编程	所有权、借用检查、trait 系统、`no_std` 支持	可构建内存安全的存储框架，类型系统可编码安全能力

2.1　比赛题目分析

2.1.1　赛题背景与核心要求

全国大学生计算机系统能力大赛操作系统设计赛功能挑战赛道要求参赛队在星绽操作系统（Asterinas）上使用 Rust 编程语言实现 `dm-crypt` 与 `dm-verity` 功能。赛题明确提出了五项技术指标：

1. `dm-verity` 基础功能：构建 SHA-256 哈希树，实现数据块完整性验证，支持内核启动参数配置；

2. dm-crypt 基础功能：透明加解密 I/O 路径，作为后端支撑 cryptsetup 对加密卷的创建和管理；
3. 根文件系统加密：使用 dm-crypt 加解密根文件系统；
4. 性能与启动要求：I/O 吞吐及延迟接近 Linux，dm-verity 不显著增加内核启动时间；
5. 技术探索：探索基于 Rust OS 的可信存储性能优化技术。

这些要求覆盖了功能完整性、Linux 兼容性、性能可观测性和技术创新性四个维度。本项目在全部五个指标上均已完成并通过验证。

2.1.2 赛题特征解析

根据赛题原文，本赛题具有以下关键特征，直接决定了项目的技术路线：

特征维度	赛题要求	本项目响应
硬件环境	服务器或个人 PC	在标准 x86_64 开发机上完成开发和测试
测试环境	基于 QEMU 虚拟机，分别部署 Linux 与星绽	使用 QEMU/KVM 运行 Asterinas，virtio-blk 设备
存储接口	virtio-blk 标准化虚拟化接口	aster-dm 通过 BIO 路径直接对接 virtio-blk
功能验证	启动参数验证 dm-verity；构造篡改验证失败	R03/V01–V07 全部 PASS，篡改后 integrity-rejected
性能验证	FIO、filebench、SQLite；boot 时间对比	五轮矩阵全部完成，启动开销 $< 3\%$

2.1.3 为什么这个赛题具有挑战性

1. **平台差异的根本性**：Asterinas 采用 Framekernel 架构，其块设备 I/O 接口（BIO 系统）与 Linux 内核完全不同。Linux 的 struct bio 和 request_queue 机制在 Asterinas 中不存在，无法直接移植代码，需要从头设计适配层。
2. **密码学工程的精确性要求**：AES-256-XTS 和 SHA-256 必须与 Linux 内核的 aes-xts-plain64 逐字节兼容。任何微小的差异（如 IV 生成的小端序处理、XTS 微调乘法的实现）都会导致 cryptsetup 无法解密 Linux 创建的加密卷，破坏互操作性。
3. **内核集成的系统复杂度**：Device Mapper 不是一个独立模块，它需要与块设备层（BIO 提交与完成回调）、设备文件系统（/dev/mapper/control）、启动流程（参数解析与根切换）等多个内核子系统协同工作，集成面广、耦合度高。
4. **性能优化的安全性约束**：所有优化（缓存、缓冲池、硬件加速）必须在不削弱安全语义的前提下进行。例如，哈希缓存必须遵循“先认证、后插入”原则，否则可能

破坏 dm-verity 的完整性信任链。

5. **可信存储的信任链建立**：`dm-verity` 的根哈希需要通过安全渠道注入（启动参数），但启动参数本身可能被篡改。这要求在系统设计中明确信任边界，并确保即使缓存命中也不会绕过根哈希验证。

2.1.4 赛题与操作系统课程知识的关联

本赛题紧密关联操作系统课程的核心知识模块，体现了理论与实践的结合：

- **I/O 系统**：块设备驱动、BIO 提交与完成、同步/异步 I/O 模型；
- **文件系统**：ext2 布局、超级块结构、`pivot_root` 根切换语义；
- **进程管理**：内核线程、中断上下文、抢占禁用与同步原语；
- **内存管理**：内核堆分配、DMA 缓冲区、零拷贝 I/O；
- **密码学工程**：分组密码模式（XTS）、哈希算法（SHA-256）、密钥管理。

2.2 相关技术调研

2.2.1 Linux Device Mapper 框架深度分析

Linux 内核的 Device Mapper 是一个通用的块设备映射框架，通过 `dmsetup` 工具和 `libdevmapper` 库提供用户态控制接口。其核心机制包括：

1. **映射表（Mapping Table）**：由一系列目标（target）组成，每个目标描述一段逻辑地址范围映射到下层设备的方式。表格式为：`<start_sector> <length> <target_type> <target_args>`；
2. **目标驱动（Target Driver）**：每个 target 类型（如 `crypt`、`verity`、`linear`）对应一个内核模块，实现 `map` 函数处理 BIO。`dm-crypt` 和 `dm-verity` 是其中应用最广泛的两种 target；
3. **控制接口**：通过 `/dev/mapper/control` 的 ioctl 命令（`DM_DEV_CREATE`、`DM_TABLE_LOAD`、`DM_DEV_SUSPEND`、`DM_DEV_STATUS`、`DM_DEV_REMOVE`）完成 mapper 的生命周期管理；
4. **用户态工具链**：`cryptsetup` 是 `dm-crypt` 的主要用户，它封装了 LUKS（Linux Unified Key Setup）格式——PBKDF 密钥派生、keyslot 管理和 LUKS header 解析均在用户态完成，内核只接收最终的 volume key 和 dm-crypt table。

本项目参考了 Linux `dm-crypt` 和 `dm-verity` 的以下设计要点，并确保实现与其语义一致：

- `dm-crypt`：XTS-AES 模式（NIST SP 800-38E），IV 生成采用 `plain64` 策略（64 位扇区号小端序），支持 `iv_offset` 和 `data_offset` 参数；
- `dm-verity`：SHA-256 构建 Merkle 树（块大小 4096 字节，哈希值 32 字节），根哈希作为信任锚，支持 on-disk 哈希树布局（每 128 个哈希值组成一个 4096 字节的

哈希块），只读语义。

2.2.2 星绽操作系统（Asterinas）架构深入分析

Asterinas 是一个采用 Framekernel 架构的 Rust 操作系统，其核心特点包括：

- **OSTD 框架**：提供内核开发所需的基础设施，包括内存管理、任务调度、同步原语、块设备抽象和中断处理；
- **块设备层（`aster-block`）**：通过 `BlockDevice` trait 和 BIO（Block I/O）系统支持 `virtio-blk` 等真实设备。BIO 是异步提交的，通过 `enqueue` + 回调完成，这与 Linux 的 `submit_bio` 类似但接口不同；
- **组件化设计**：内核功能以组件（component）形式组织，通过 `Components.toml` 声明依赖，支持按需编译。本项目将 `aster-dm` 作为独立组件接入；
- `no_std` 环境：内核代码运行在无标准库的环境中，依赖 `alloc` 和核心库，这要求谨慎管理动态内存分配；
- **启动流程**：内核解析命令行参数后，通过 initramfs 中的 `init` 脚本完成根文件系统挂载和 `pivot_root` 切换。

本项目对 Asterinas 的接入采用补丁序列(`asterinas_port/patches/0001--0007`)方式，每个补丁对应一个独立可验证的增量功能，确保集成过程可审计、可回溯。

2.2.3 密码学标准与算法调研

AES-256-XTS 模式（NIST SP 800-38E）

XTS-AES 模式是专为存储设备加密设计的分组密码操作模式，其核心特性包括：

- 使用两个独立的 AES-256 密钥：一个用于数据块加密（Data Key），一个用于微调生成（Tweak Key）；
- 每个 128 位数据块使用 128 位微调值（由扇区号和块内偏移经过 AES 加密派生）；
- 支持并行加密/解密，适合块设备的随机 I/O 模式；
- 与 CBC 模式相比，XTS 的每个扇区独立加密，一个扇区的损坏不影响其他扇区，更适合存储设备。

Linux 内核的 `aes-xts-plain64` 使用 64 位扇区号作为 IV 的基础，以小端序编码后经 Tweak Key 加密生成微调值，然后对每个 16 字节块进行 XTS 变换。本项目在 `crypto.rs` 中独立实现了该算法，实现细节包括：

1. 将 64 位扇区号编码为 16 字节小端序（高 8 字节为零），经 Tweak Key 加密生成初始微调值；
2. 对扇区内每个 16 字节块，先用当前微调值异或，再经 Data Key 加密（或解密），再与微调值异或；
3. 每个块处理后，微调值在 GF(2^{128}) 中乘以 x（即左移 1 位，若溢出则异或 0x87）。

该实现通过了与 Linux 内核逐字节比对验证，固定向量 SHA-256 为 `4334c17f...`。

SHA-256 哈希算法（FIPS PUB 180-4）

SHA-256 是 `dm-verity` 使用的哈希算法，输出 32 字节（256 位）摘要。其核心计算包括：

- 消息填充：先补 1 位，再补 0 位至长度模 512 为 448，最后附加 64 位原始长度；
- 64 轮迭代压缩函数，每轮使用 8 个 32 位工作变量和 64 个 32 位常数；
- 最终输出 8 个 32 位字的级联。

本项目在 `sha256.rs` 中实现了无外部依赖的纯 Rust SHA-256，通过标准测试向量（空字符串、`"abc"`）验证正确性。

2.2.4 Rust 在可信存储领域的独特优势

选择 Rust 而非 C 实现可信存储模块，具有以下独特优势：

1. **内存安全**：Rust 的所有权和借用检查在编译期杜绝了悬垂指针、use-after-free 和缓冲区溢出——这些正是 C 内核中安全漏洞的主要来源（据统计约占 Linux 内核漏洞的 65%）。在可信存储场景中，内存安全漏洞可能导致密钥泄露或完整性绕过。
2. **类型系统编码安全属性**：Rust 的类型系统可以将安全属性编码到类型中。例如，本项目中的 `TargetCapabilities` 结构体使组合栈的安全能力（confidentiality、integrity、read_only）可在编译期静态检查。
3. **零成本抽象**：Rust 的 trait 和泛型不会引入运行时开销，使得抽象（如 `BlockDevice`）可以像手写 C 代码一样高效，同时保持代码可复用性。
4. `Send` 和 `Sync` 的自动推导：Rust 通过这两个标记 trait 在编译期保证线程安全，避免了 C 内核中因锁使用不当导致的竞态条件。
5. **更安全的错误处理**：`Result<T, E>` 强制开发者显式处理错误，避免了 C 内核中因错误码被忽略而导致的静默失败。

本项目正是 Rust 在可信存储领域优势的实践验证——我们使用相同的核心算法（AES-XTS、SHA-256）实现了与 Linux 兼容的可信存储，同时获得了 Rust 提供的安全保障。

2.2.5 参考实现与开源生态调研

名称	来源/位置	参考内容与启示
Linux 内核 dm-crypt	`drivers/md/dm-crypt.c`	XTS 实现细节、plain64 IV 生成、table 格式解析
Linux 内核 dm-verity	`drivers/md/dm-verity.c`	Merkle 树 on-disk 布局、验证逻辑、缓存策略
cryptsetup 工具	`https://gitlab.com/cryptsetup/cryptsetup`	用户态控制协议、LUKS 格式、plain 模式 open/close
RustCrypto `aes`	`https://crates.io/crates/aes`	AES 算法接口设计参考（本项目为独立实现）
RustCrypto `xts-mode`	`https://crates.io/crates/xts-mode`	XTS 模式 API 参考（本项目为独立实现）
NIST SP 800-38E	`nvlpubs.nist.gov/nistpubs/Legacy/SP/`	XTS-AES 模式标准定义
FIPS PUB 180-4	`nvlpubs.nist.gov/nistpubs/FIPS/`	SHA-256 算法标准
Asterinas 源码	`https://github.com/asterinas/asterinas`	OSTD 框架、BIO 接口、组件系统
Asterinas ATC'25 论文	USENIX ATC 2025	Framekernel 架构、TCB 最小化设计理念
《The Rust Programming Language》	`https://doc.rust-lang.org/book/`	Rust 系统编程、`no_std` 开发

2.2.6 兼容性验证策略调研

为确保与 Linux 生态的互操作性，本项目设计了分层兼容性验证策略：

1. **算法层**：固定向量测试——AES-XTS 已知明文测试向量、SHA-256 标准测试向量（空字符串、`"abc"`）；
2. **协议层**：与 Linux 内核 dm-crypt 逐字节比对——同一密钥（512 位）、同一明文（0x5a 填充的一个扇区）、同一扇区号（扇区 0）生成的密文 SHA-256 一致；
3. **工具层**：在 Asterinas KVM 中运行标准 `cryptsetup` 工具（plain 模式），验证 `open`、`dd read`、`close` 完整流程（R01）；
4. **性能层**：使用与 Linux 相同的 fio 参数和 workload，在同一 backing image 上分别测试，确保可比性；

5. **正确性层**：通过物理篡改（单字节翻转）验证 dm-verity 的检测能力，确保完整性边界未被削弱（V03–V07）。

2.3 技术可行性评估

基于上述调研，本项目从以下维度评估技术可行性：

维度	风险评估	应对措施
算法正确性	**低风险**：AES-XTS 和 SHA-256 有明确标准和参考实现	固定向量测试 + Linux 逐字节比对
Linux 兼容性	**中风险**：plain64 IV 和 XTS 实现细节易出错	早期编写交叉验证脚本，迭代修正
Asterinas 集成	**中风险**：BIO 接口和启动流程需要深入理解	补丁序列渐进式集成，每步独立验证
性能达标	**中风险**：虚拟机环境 I/O 性能波动大	固定五轮、交替顺序、保留原始数据
优化不削弱安全	**高风险**：缓存可能绕过完整性校验	“先认证后插入”设计原则 + 计数器验证

项目的总体可行性判断为**高可行**，理由如下：

1. 核心算法有明确的标准（NIST、FIPS）和参考实现（Linux 内核），验证手段成熟；
2. 双目标架构（用户态 + 内核态）使用户态可作为“安全网”，在进入内核前充分验证算法和逻辑；
3. Asterinas 提供了清晰的组件接入接口（`Components.toml`、`aster-cmdline`、`aster-block`），集成路径明确；
4. 性能测试合同（`benchmark_contract.md`）固定了环境、参数、轮数和统计口径，结果可复现、可审计。

2.4 小结

本章从赛题要求、赛题特征、技术挑战、课程关联、相关技术调研、Rust 独特优势和可行性评估七个维度对比赛题目进行了全面分析。

核心调研结论包括：

1. Linux Device Mapper 框架提供了成熟的 dm-crypt/dm-verity 参考实现，但 Asterinas 的内核接口与之不同，需要从头实现 Rust 适配层；
2. AES-256-XTS 和 SHA-256 有明确的标准规范，通过固定向量和 Linux 逐字节比对可验证兼容性；

3. Rust 的所有权系统、类型系统和零成本抽象为可信存储模块提供了 C 语言无法比拟的内存安全和可组合性；
4. 双目标架构（用户态验证 + 内核态集成）和分层兼容性验证策略确保了项目的可验证性和可复现性；
5. 赛题要求的 QEMU + virtio-blk 标准化测试环境已在本项目中完整实现并用于所有验证。

基于统一抽象、渐进式集成和可观测性内置的技术路线，项目具备充分的可行性和可验证性。后续章节将详细阐述基于这些分析得出的系统架构设计。

第 3 章　系统框架设计

本章框架设计速览

设计维度	核心决策	技术收益
核心抽象	`BlockDevice` trait（约 150 行接口定义）	统一所有存储后端和映射器，实现可互换、可堆叠
架构模式	装饰器模式 + 组合模式	支持任意层级的映射器链（Passthrough → Crypt → Verity）
双目标复用	核心逻辑与平台无关	用户态验证算法正确性，内核态保证真实性能，共享同一套核心实现
线程安全	Rust `Send` + `Sync` + 自旋锁	编译期保证线程安全，避免竞态条件
安全能力传播	`TargetCapabilities` 类型编码	组合栈的安全属性在编译期静态检查
性能可观测	原子计数器 + 阶段计时	精确量化每个子系统的开销，指导优化方向

3.1　设计原则

本项目的基本设计原则是使用统一的抽象来表示任何能够进行块级 I/O 的实体，并采用“分层”或“可组合”的架构，使加密和校验逻辑与底层存储介质完全解耦。核心设计原则包括：

1. **统一抽象**：所有存储后端和映射器均实现同一个 `BlockDevice` trait，确保模块可互换、可堆叠；
2. **双目标共享**：核心逻辑与平台无关，同时服务于用户态命令行工具（`dmctl`）和星绽内核（`aster-dm`）；

3. **安全优先**：所有优化（缓存、缓冲池）遵循“先认证后使用”原则，不削弱安全边界；
4. **可观测性内置**：关键路径嵌入原子计数器和阶段计时，支持性能归因和优化验证。

这些原则共同确保了系统的可扩展性、可验证性和安全性——这正是 Device Mapper 作为可信存储底座的核心质量要求。

3.2 核心抽象：BlockDevice Trait

`BlockDevice` trait 是库中所有存储操作的基础接口，定义了用于读写固定大小数据块的同步接口。为了支持跨线程的并发访问并安全地集成到星绽内核或用户态工具中，该 trait 要求实现 `Send` 和 `Sync` 标记。

```
pub trait BlockDevice: Send + Sync {
    fn block_size(&self) -> usize;
    fn num_blocks(&self) -> u64;
    fn read_block(&self, block_id: u64, buf: &mut [u8]) ->
    Result<()>;
    fn write_block(&self, block_id: u64, buf: &[u8]) ->
    Result<()>;
}
```

所有 I/O 方法均返回 `Result<()>`，其中错误被封装在 `DmError` 枚举中，涵盖无效块大小、无效块号、密钥错误、完整性违规等场景。这个 trait 的设计借鉴了 Linux 内核 `block_device_operations` 的职责分离思想，但通过 Rust 的类型系统提供了更强的安全性保证。

在 Asterinas 内核中，该 trait 进一步映射到 `aster-block` 组件的 BIO 接口，从而支持真实的 VirtIO 块设备。

3.3 可组合的映射器链模式

该架构采用类似装饰器（Decorator）的设计模式。每个映射器（如 `DmCryptDevice` 和 `DmVerityDevice`）既实现了 `BlockDevice` trait，又在内部消费（包装）另一个实现了 `BlockDevice` 的对象，支持功能的任意堆叠与组合。

例如，一个典型的用户态栈可能是：

```
let file = FileBlockDevice::open("disk.img")?;
let crypt = DmCryptDevice::new(file, key)?;
```

```
let verity = DmVerityDevice::open(crypt, tree, root_hash)?;
```

在星绽内核中，这种组合通过 DmDevice 结构体实现，它根据 DmTarget 枚举（Passthrough、Crypt、Verity）分发 BIO 请求。由于每个 mapper 的下层是 Arc<dyn BlockDevice>，启动阶段可将 rootfs_verity.device 指向已注册的 crypt mapper 名（如 dm-crypt0），或显式设置 dm_stack=crypt,verity，从而形成与用户态相同的叠栈：vdc → dm-crypt0 → dm-verity0。

3.4 核心模块分解

项目被结构化为离散的模块，以清晰地分离密码学原语、数据结构和设备逻辑。整个 dm crate 约 3000 行 Rust 代码，模块划分如下：

表 3.1: 核心模块职责划分与代码量

模块	主要职责	代码量（约）
block_device	基础 I/O 抽象与物理/内存后端	400 行
dm_crypt	使用 AES-XTS 的透明加密逻辑	350 行
dm_verity	只读完整性校验逻辑	200 行
hash_tree	Merkle 树的构建与管理	150 行
sha256	纯 Rust SHA-256（无外部依赖）	120 行
device_mapper	设备链接（chaining）的基础设施	180 行
statistics	I/O 性能计数器与阶段归因	150 行
runtime	运行时 mapper 管理（Linux dm_ioctl 兼容）	260 行
error	统一错误处理	80 行

BlockDevice 实现涵盖了多种后端场景：

- MemoryBlockDevice：基于内存向量的设备，主要用于单元测试和易失性存储；
- FileBlockDevice：由宿主机文件系统上的文件支持的持久化设备，由 dmctl 工具使用；
- PassthroughDevice：将调用直接转发给内部设备的包装器，用于验证映射器链；
- VirtioBlkAdapter：（星绽移植版）将内核特定的块设备句柄适配到 BlockDevice 接口，位于 aster-dm 组件中。

3.5 双目标架构：跨环境可移植性

一个核心设计目标是：相同的逻辑同时服务于用户态演示和星绽内核。这是通过严格隔离环境特定代码来实现的。

- **用户态**：dmctl 工具实例化一个 FileBlockDevice，并根据命令行参数将其包装在 DmCryptDevice 或 DmVerityDevice 中；所有 I/O 操作是同步的，使用 std::sync::Mutex 保证线程安全。
- **内核态**：在星绽中，VirtioBlkAdapter 包装了内核内部的块设备句柄。由于该适配器实现了 BlockDevice，DmCryptDevice 可以无需任何修改地直接包装它；I/O 操作是异步的（BIO），使用 spin::Mutex 和原子操作保证并发安全，在 AES-NI 变换期间还须通过 disable_preempt 和 irq::disable_local 保护 XMM 寄存器。
- no_std 兼容性：核心 dm 库设计为最小化依赖（例如，aes crate 设置 default-features = false），以促进在可能限制标准库的内核环境中执行。

代码实体映射关系如下：

表 3.2: 用户态与内核态实体映射

功能	用户态（dmctl）	内核态（aster-dm）
块设备后端	FileBlockDevice / MemoryBlockDevice	VirtioBlkAdapter（包装 virtio-blk）
加密映射器	DmCryptDevice	DmTarget::Crypt
完整性映射器	DmVerityDevice	DmTarget::Verity
组合构建器	DmStackBuilder	dm_stack 启动参数（补丁 0007）
控制接口	N/A（用户态直接调用）	/dev/mapper/control（dm_ioctl）
同步原语	std::sync::Mutex	spin::Mutex + disable_preempt

3.6 数据流架构

在此模型中，DmCryptDevice 在执行 AES-256-XTS 转换后，将生成的密文传递给链中的下一个设备。类似地，DmVerityDevice 拦截读请求，以根据 Merkle 树校验数据的完整性。

3.6.1 dm-crypt 数据流

写入路径：

1. 应用程序或文件系统向 DmCryptDevice 发起 write_block 请求；
2. 根据逻辑块 ID 和 iv_offset 计算对应的物理扇区号；
3. 从 scratch pool 获取临时缓冲区，调用 Aes256XtsCipher 对数据进行加密，使用扇区号作为 plain64 IV；
4. 通过 BioSegment::alloc_bytes 创建 bounce segment，将密文写入底层 BlockDevice；
5. 底层设备完成写入后，释放 scratch 缓冲区，完成上层 BIO。

读取路径是上述过程的逆操作：从底层设备读取密文到 scratch 缓冲区，使用相同的扇区号进行解密，然后写回上层 segment，最后释放 scratch 缓冲区。

3.6.2 dm-verity 数据流

读取路径：

1. 从底层块设备读取原始数据块；
2. HashTree::verify_block 计算检索到的数据的 SHA-256 哈希值；
3. 在每一层，从 on-disk 哈希树或认证后缓存中检索对应哈希值；
4. 将重新计算出的哈希值与 DmVerityDevice 中存储的 root hash 进行比对；
5. 如果哈希值不匹配，返回 IntegrityViolation 错误；若匹配且哈希块此前未缓存，则将其插入认证后缓存供后续复用。

3.7 线程安全保证

所有后端均实现了 Sync，允许跨线程共享：

- MemoryBlockDevice：在整个块向量周围使用 Mutex，确保安全性但将所有 I/O 串行化；
- FileBlockDevice：在 File 对象周围使用 Mutex，锁在 seek、read/write 和 flush 序列的整个持续时间内被持有；
- PassthroughDevice：依赖于底层设备实现了 Sync。

在星绽内核中，DmDevice 通过 enqueue 方法处理 BIO，内部使用 spin::Mutex 保护共享状态（如 CryptScratchPool 和 VerifiedHashBlockCache），并遵循内核的并发模型。AES-NI 变换期间通过 disable_preempt 和 irq::disable_local 确保 XMM 寄存器不被中断上下文破坏——这是因为 AES-NI 指令使用 XMM 寄存器，而中断处理程序可能复用这些寄存器导致数据损坏。

3.8 性能可观测性设计

为了支持性能归因和优化验证，架构中集成了统计模块：

- InstrumentedDevice：一个装饰器，可以插入到任意两层之间，记录读/写次数、字节数、失败次数以及累计和最大延迟（以纳秒计）；

- IoStatsSnapshot：提供稳定的快照，便于在基准测试前后对比；
- 在 Asterinas 内核中，通过启动参数 dm.perf_stats=1 启用细粒度阶段计时（如 lower_wait、AES-XTS、SHA-256、cache_lookup 等），并通过 /proc/dm-performance-stats 导出；
- /proc/dm-verity-stats 提供 verified、bio_blocks、data_rereads、hash_hits、hash_misses、auth_shortcuts 等计数器，直接反映优化效果——这些计数器在正式性能测试中用于验证 BIO 复用（data_rereads=0）和缓存命中（auth_shortcuts）的实际收益。

  这种设计使得开发者可以精确量化每个映射器层的开销，从而指导优化方向。

3.9 运行时控制面（Linux 兼容）

为了支持 cryptsetup 等用户态工具，项目实现了与 Linux dm_ioctl 兼容的控制接口：

- DmControlDevice：在 /dev/mapper/control 提供 ioctl 处理，支持 DM_DEV_CREATE、DM_TABLE_LOAD、DM_DEV_SUSPEND、DM_DEV_STATUS、DM_DEV_REMOVE 等命令；
- RuntimeMapper：管理动态创建的 mapper，包括状态机（Empty、Loaded、Active）和密钥的清除——当 mapper 被删除时，未激活表中的密钥会被显式清零，防止密钥残留在内存中；
- 通过该接口，Asterinas 内核可以接受来自 cryptsetup 的标准 table 字符串，动态激活加密映射器，从而实现与 Linux 生态的互操作。

3.10 安全能力传播

DmStackBuilder 和 DmStack 提供安全能力的自动汇总：

```
pub struct TargetCapabilities {
    pub confidentiality: bool,
    pub integrity: bool,
    pub read_only: bool,
}
```

每个 target 声明自己提供的安全能力，最终栈通过 fold 操作自动汇总，使上层调用者可以静态获知组合后的安全属性。例如，crypt + verity 栈自动获得 confidentiality=true、integrity=true、read_only=true。这种设计将安全策略的传播在编译期完成，避免了运行时动态检查的开销和遗漏风险。

3.11 Asterinas 内核集成架构

aster-dm 组件通过以下方式接入 Asterinas 内核：

1. **组件声明**：在 Components.toml 中注册 aster-dm，在 kernel/Cargo.toml 中引入依赖；
2. **启动参数解析**：使用 aster-cmdline::define_kv_param! 宏定义 dm_crypt.*、rootfs_verity.* 和 dm_stack 参数；
3. **块设备注册**：通过 aster_block::register 将 mapper 注册为块设备，分配主设备号和次设备号；
4. **BIO 处理**：在 BlockDevice::enqueue 中根据 DmTarget 类型分发到对应的处理函数（process_crypt、process_verity），所有 I/O 操作均通过 BIO 异步提交；
5. **根切换**：initramfs 中的 init 脚本在 dm-root 模式下挂载 mapper，执行 pivot_root 切换到受保护根。

补丁序列（asterinas_port/patches/0001--0007）记录了完整的集成演进历史，每一补丁对应一个可独立验证的增量功能，总补丁代码量约 4000 行。

3.12 小结

本章详细阐述了系统的核心架构，包括统一的 BlockDevice 抽象、可组合的映射器链模式、模块分解、双目标跨环境可移植性、数据流、线程安全保证、性能可观测性设计、运行时控制面和 Asterinas 内核集成架构。通过将核心逻辑与平台细节解耦，并增加性能可观测性和安全能力传播机制，该项目为后续的加密、完整性校验及性能优化奠定了坚实的基础。

关键设计亮点回顾：

- BlockDevice trait 作为统一抽象，约 150 行接口定义支撑了整个存储安全框架；
- 双目标架构使用户态可作为“安全网”快速验证算法，内核态保证真实 I/O 路径和性能；
- Rust 的类型系统（Send、Sync、TargetCapabilities）在编译期保证了线程安全和安全能力传播的正确性；
- 性能可观测性设计将优化决策建立在量化数据之上，而非经验猜测。

在接下来的章节中，我们将深入探讨 dm-crypt 和 dm-verity 的具体实现细节，以及如何将上述框架设计落地为可运行的代码。

第 4 章　开发计划

本章开发计划速览

阶段	时间	核心内容	关键交付物
第一阶段	06-23 – 06-28	用户态核心实现	`dm` crate、单元测试、`dmctl` 工具
第二阶段	06-29 – 07-10	Linux 兼容性与工具链	XTS 兼容验证、`cryptsetup` plain 模式
第三阶段	07-11 – 07-20	内核补丁与 BIO 集成	补丁 0001–0004、rootfs 切换
第四阶段	07-21 – 07-31	性能优化与基准框架	补丁 0005–0006、FIO 矩阵脚本
第五阶段	08-01 – 08-11	组合栈与最终验证	补丁 0007、五轮性能矩阵、证据归档
第六阶段	08-09 – 08-14	文档整理与答辩准备	技术文档、答辩 PPT、设计赛道文档



图 4.1: 项目开发甘特图（2026-06-23 至 2026-08-14）

4.1　开发方法论

在制定开发计划时，我们遵循以下方法论原则，确保开发过程可控、可追溯：

1. **先验证、后集成**：所有核心算法（AES-XTS、SHA-256、Merkle 树）先在用户态充分验证，再通过补丁序列接入内核。用户态作为“安全网”，可以在不依赖复杂内核环境的前提下快速迭代和调试；
2. **渐进式功能增量**：每个补丁对应一个独立可验证的功能增量（补丁 0001：组件骨架；0002：BIO 路径；0003：根切换；0004：控制面；0005：性能优化；0006：阶段归因；0007：组合栈），每一步都有明确的验收标准；
3. **测试驱动验证**：每个功能模块完成后立即编写对应的测试用例（单元测试、兼容性测试、KVM 正确性测试），确保证据链完整；
4. **性能基准先行**：在优化之前先建立性能基准（五轮 FIO、Filebench、SQLite、启动耗时），优化措施的效果通过前后对比量化验证；
5. **文档与代码同步**：每个阶段完成后同步更新技术文档和用户手册，避免“代码改完、文档滞后”的常见问题。

4.2 阶段划分与时间安排

本项目实际开发周期为 2026 年 6 月 23 日至 8 月 14 日，共计 50 天，分为六个阶段。以下按阶段详细描述计划内容与实际完成情况。

4.3 第一阶段：用户态核心实现

4.3.1 目标

在用户态完成 Device Mapper 核心逻辑，不依赖任何操作系统特定接口，使用 `MemoryBlockDevice` 和 `FileBlockDevice` 进行验证。这是整个项目的“地基”阶段。

4.3.2 任务分解

1. 设计并实现 `BlockDevice` trait 及内存/文件后端；
2. 实现 `sha256` 模块（纯 Rust，无外部依赖，约 120 行）；
3. 实现 `hash_tree` 模块（Merkle 树构建与验证，约 150 行）；
4. 实现 `DmCryptDevice`（AES-256-XTS，使用 `aes` crate，约 350 行）；
5. 实现 `DmVerityDevice`（只读完整性校验，约 200 行）；
6. 实现 `DmStackBuilder`（可组合映射器链，约 180 行）；
7. 编写单元测试覆盖核心场景（13+ 个测试）；
8. 实现 `dmctl` 命令行工具的基础子命令（约 800 行）。

4.3.3 交付物与验收标准

- **交付物**：`crates/dm/src/` 全部核心模块；`crates/dmctl/src/main.rs` 基础命令
- **验收标准**：
  1. `demo-crypt` 输出 `ciphertext_differs: true`；
  2. `demo-verity` 输出 `tamper_detected: true`；
  3. 所有单元测试 PASS。

4.3.4 实际完成情况

- 完成时间：2026-06-23（Git commit `40a52179` `feat: add Linux-compatible device mapper implementation`）；
- 通过全部 13+ 个单元测试；
- `demo-crypt` 和 `demo-verity` 验证通过；
- 代码量：约 1500 行 Rust。

4.4 第二阶段：Linux 兼容性与工具链

4.4.1 目标

确保 AES-256-XTS 实现与 Linux 内核 `aes-xts-plain64` 逐字节兼容，并为 `cryptsetup` 工具提供控制接口。这是确保项目与 Linux 生态互操作的关键阶段。

4.4.2 任务分解

1. 实现 `DmCryptTable` 解析器（支持 Linux table 格式，约 200 行）；
2. 实现 `Aes256XtsCipher` 的 `plain64` IV 语义；
3. 编写 `verify_linux_xts_compat.sh` 交叉验证脚本；
4. 实现 `/dev/mapper/control` 的 `dm_ioctl` 兼容接口（`dm_control.rs`，约 520 行）；
5. 实现 `RuntimeMapper` 状态机（Empty、Loaded、Active，约 260 行）；
6. 在 KVM 中验证 `cryptsetup open/read/close` 流程（R01）。

4.4.3 交付物与验收标准

- **交付物**：`DmCryptTable` 和 `DmCryptDevice::from_table`；`kernel/src/device/misc/dm_control` `kernel/comps/dm/src/runtime.rs`
- **验收标准**：
  1. `verify_linux_xts_compat.sh` 输出 `PASS`，SHA-256 匹配；
  2. KVM 中 `cryptsetup open/read/close` 全部 PASS。

4.4.4 实际完成情况

- 完成时间：2026-07-14（补丁 0004）；
- Linux XTS 固定向量 SHA-256：`4334c17f...`；
- R01 cryptsetup smoke PASS（证据：`R01-cryptsetup-persistent-cache.log`）；
- 代码量：新增约 1000 行内核代码。

4.5 第三阶段：内核补丁与 BIO 集成

4.5.1 目标

将 `aster-dm` 组件完整接入 Asterinas 内核，支持启动参数解析、mapper 注册、BIO 处理和根文件系统切换。这是项目从“用户态玩具”走向“真实内核模块”的关键阶段。

4.5.2 任务分解

1. 补丁 0001：组件骨架、crypt/verity target、VirtIO 适配；
2. 补丁 0002：启动参数解析、mapper 注册、ext2 魔数自检；
3. 补丁 0003：initramfs 挂载与 `pivot_root` 切换；
4. 补丁 0004：`cryptsetup` 控制面与 mapper 生命周期；
5. 验证 dm-crypt 根文件系统启动（R02）；
6. 验证 dm-verity 根文件系统启动（R03/V01）；
7. 验证单字节篡改拒绝（V02–V07）。

4.5.3 交付物与验收标准

- **交付物**：`asterinas_port/patches/0001--0004.patch`；`tools/run_dm_crypt.sh` 和 `tools/run_dm_verity.sh`
- **验收标准**：
  1. R02 串口输出 `PASS system root switched to protected mapper`；
  2. R03/V01 串口输出 `status=verified` 和根切换 PASS；
  3. V02–V07 篡改后输出 `status=integrity-rejected`，无根切换 PASS。

4.5.4 实际完成情况

- 完成时间：2026-07-14（补丁 0001–0004）；
- R01–R03、C01–C05、V01–V07 全部 PASS；
- 证据日志位于 `docs/evidence/stabilization/correctness-kvm/`；

- 补丁代码量：约 2500 行。

4.6 第四阶段：性能优化与基准框架

4.6.1 目标

实现性能优化（AES-NI、scratch pool、hash cache、BIO 复用），并建立可复现的五轮性能基准框架。本阶段的成果直接影响“性能与启动要求”指标的达成。

4.6.2 任务分解

1. 补丁 0005：AES-NI 后端、scratch pool、hash cache、正式 benchmark 参数；
2. 补丁 0006：性能阶段归因（`/proc/dm-performance-stats`）与正式合同；
3. 实现 `scripts/run_fio_matrix.sh`（Linux host 和 Asterinas KVM）；
4. 实现 Filebench、SQLite、启动耗时测试脚本；
5. 完成 A/B 对比验证优化效果。

4.6.3 交付物与验收标准

- **交付物**：补丁 0005–0006；`scripts/run_fio_matrix.sh`、`scripts/run_filebench_matrix.sh`、`scripts/run_sqlite_matrix.sh`、`scripts/run_boot_matrix.sh`
- **验收标准**：
  1. 五轮 FIO 矩阵可自动执行并生成 CSV；
  2. `dm.perf_stats=1` 时阶段计时正常输出；
  3. 优化后 `data_rereads=0`，`auth_shortcuts` > 0。

4.6.4 实际完成情况

- 完成时间：2026-08-03（补丁 0005–0006）；
- AES-NI 启动日志显示 `backend=AES-NI`；
- 正式性能矩阵于 2026-08-11 完成；
- 补丁代码量：约 800 行。

4.7 第五阶段：组合栈与最终验证

4.7.1 目标

实现 `dm-crypt` $\rightarrow$ `dm-verity` 组合栈（补丁 0007），完成全部性能基准测试并归档证据。这是项目从“单层保护”走向“组合保护”的最终形态。

4.7.2 任务分解

1. 补丁 0007：`dm_stack=crypt,verity` 启动路径；
2. 实现 `scripts/build_stack_rootfs.sh` 和 `scripts/run_dm_stack.sh`；
3. 完成 Linux host FIO 13+13 项（raw + dm-crypt）；
4. 完成 Asterinas KVM FIO 13+7 项（dm-crypt + dm-verity）；
5. 完成 Filebench、SQLite、启动耗时各 5 轮；
6. 运行 `check_benchmark_package.py` 验证包完整性。

4.7.3 交付物与验收标准

- **交付物**：补丁 0007；`scripts/build_stack_rootfs.sh` 和 `scripts/run_dm_stack.sh`
- **验收标准**：
  1. 组合栈串口输出 `stacked mapper: vdc -> dm-crypt0 -> dm-verity0`；
  2. 最终矩阵包含 46 个 PASS 单元格；
  3. `review_release_check.py` 全部通过。

4.7.4 实际完成情况

- 完成时间：2026-08-11（补丁 0007，commit `6385e70b`）；
- 最终矩阵 46 PASS + 6 N/A；
- 证据目录完整，`review_release_check.py` PASS；
- 补丁代码量：约 150 行（组合栈逻辑本身精简，复用已有 crypt/verity 实现）。

4.8 第六阶段：文档整理与答辩准备

4.8.1 目标

完成全部技术文档、答辩材料、演示视频和设计赛道文档的整理与提交。

4.8.2 任务分解

1. 技术文档（LaTeX）：架构设计、模块说明、测试结果（9 章 + 附录）；
2. 设计赛道文档（LaTeX）：10 章节完整版；
3. 答辩 PPT（`presentation_project.pptx`）；
4. 演示视频（含运行环境介绍与受保护 rootfs 启动）；
5. 作品进展汇报幻灯片；
6. AI 工具使用声明。

4.8.3 交付物与验收标准

- **交付物**：`docs/document/main.pdf`；`docs/design_doc/design_doc.pdf`；`presentation_project.pptx`；作品演示视频.mp4；作品进展汇报幻灯片.pptx
- **验收标准**：
  1. 所有文档可编译生成 PDF；
  2. 演示视频完整展示受保护 rootfs 启动；
  3. AI 工具声明完整披露工具名称、模型和使用场景。

4.8.4 实际完成情况

- 完成时间：2026-08-14；
- 全部文档、PPT 和视频已提交至指定平台；
- 文档总页数：技术文档约 120 页，设计赛道文档约 80 页。

4.9 里程碑与关键交付物汇总

表 4.1: 关键里程碑与交付物

里程碑	日期	关键交付物
M1：用户态核心完成	06-23	`dm` crate、`dmctl` 工具、单元测试 PASS（`40a52179`）
M2：Linux 兼容性验证	07-10	XTS 逐字节兼容证据、`/dev/mapper/control`（补丁 0004）
M3：内核 rootfs 切换	07-14	R01–R03、C01–C05、V01–V07 全部 PASS（补丁 0001–0004）
M4：性能优化与框架	08-03	AES-NI、scratch pool、hash cache、五轮 FIO 脚本（补丁 0005–0006）
M5：组合栈与最终验证	08-11	补丁 0007、46 PASS 性能矩阵（`6385e70b`）
M6：文档与答辩准备	08-14	技术文档 PDF、答辩 PPT、演示视频、设计赛道文档

4.10 开发环境与工具链

工具/环境	版本/配置
开发语言	Rust 1.75.0+（用户态），Asterinas 工具链 `nightly-2026-04-03`（内核）
构建系统	Cargo workspace（用户态），`make kernel` + OSDK（内核）
虚拟化	QEMU 8.2.2+，KVM 加速，`-cpu host`
容器环境	Docker，镜像 `asterinas/asterinas:0.18.0-20260618`
版本控制	Git，托管于 GitLab
文档工具	LaTeX（技术文档、设计赛道文档），Microsoft PowerPoint（答辩 PPT）
性能测试	fio 3.36、Filebench 1.5-alpha3、SQLite 3.45.1

4.11 风险应对

风险	影响	应对措施
AES-XTS 与 Linux 不兼容	cryptsetup 无法互操作，影响赛题指标 2	提前编写交叉验证脚本（`verify_linux_xts_compat.sh`），逐字节比对，迭代修正直到完全一致
Asterinas BIO 接口变化	内核集成延迟	使用固定 commit 基线（`9d79542c`），补丁隔离，降低对上游 API 变动的依赖
性能测试不稳定	结果不可复现，影响赛题指标 4	固定五轮、交替顺序、保留原始 JSON/CSV，报告中位数和标准差而非选择最好一轮
虚拟化环境差异	跨平台倍率误解	分平台展示，不计算跨环境倍率，Linux 作为参考对照而非性能标尺
缓存安全性削弱	完整性边界被突破，影响赛题指标 1	“先认证后插入”设计原则，每个缓存块必须经完整 Merkle 路径验证才可插入，计数器（`auth_shortcuts`）验证
Nix 构建/网络依赖	构建失败或耗时过长	持久化 Cargo/rustup volume，预置 cryptsetup 到 initramfs，固定 Docker image digest

4.12 计划执行偏差分析

实际开发与计划整体一致，仅有以下微调：

- **第三阶段提前完成**：原计划 07-11 至 07-20，实际 07-14 完成，得益于用户态阶段的充分验证使内核集成更加顺利；
- **第四、五阶段合并执行**：性能优化与组合栈验证存在重叠，实际在 07-21 至 08-11 期间并行推进；
- **文档阶段与开发阶段重叠**：技术文档的编写从第一阶段即开始，第六阶段主要完成最后的整合与润色。

4.13 小结

本章详细阐述了项目的开发计划，包括六个阶段的划分、任务分解、交付物、验收标准、实际完成情况和风险应对。项目实际开发周期为 2026 年 6 月 23 日至 8 月 14 日，共 50 天，完成了从用户态核心实现到内核集成、性能优化、组合栈验证和文档整理的全部工作。所有阶段均按计划完成，关键里程碑均有可复核的证据支撑。

计划制定遵循“先验证、后集成、渐进式增量、测试驱动、文档同步”的方法论，确保了开发过程的可控性和可追溯性。项目最终成果全面覆盖赛题五项技术指标，代码、证据、文档和演示材料均按大赛要求提交至指定托管平台。

第 5 章　比赛过程中的重要进展

本章关键进展速览

阶段	时间	核心成果	验证状态
用户态核心	06-23–06-28	dm crate + `dmctl` 工具，约 1500 行 Rust 代码	13+ 单元测试 PASS
Linux 兼容性	06-29–07-10	AES-XTS 逐字节兼容，`/dev/mapper/control` 控制面	固定向量 SHA-256 匹配
内核集成	07-11–07-20	补丁 0001–0004，BIO 路径 + rootfs 切换 + crypt-setup 控制	12 组 KVM 测试全部 PASS
性能优化	07-21–07-31	AES-NI、scratch pool、hash cache、阶段归因	启动日志显示 AES-NI
组合栈与验证	08-01–08-11	补丁 0007，46 PASS 性能矩阵，启动开销 <3%	`review_release_check.py` PASS
文档与答辩	08-09–08-14	技术文档 PDF、答辩 PPT、演示视频、设计赛道文档	全部提交完成



图 5.1: 项目进展时间线总览

5.1　概述

本章按时间顺序记录项目从开始到完成过程中的关键里程碑和技术突破。所有进展均有对应的 Git 提交记录、补丁文件或证据日志支撑，反映了项目的真实演进路

径。

项目开发周期为 2026 年 6 月 23 日至 8 月 14 日，共计 50 天。在此期间，项目经历了从零开始搭建核心框架、验证 Linux 兼容性、完整接入 Asterinas 内核、进行性能优化、实现组合栈保护，到最后完成文档和答辩材料的全流程。每一阶段均有可复核的代码、测试日志和性能数据作为支撑。

5.2 第一阶段：用户态核心实现（2026-06-23 至 2026-06-28）

5.2.1 6 月 23 日：初始提交与核心框架搭建

关键事件：

- 完成项目仓库初始化，提交核心 `dm` crate 框架（commit `6bb9bc92`、`40a52179`）；
- 实现 `BlockDevice` trait 及 `MemoryBlockDevice`、`FileBlockDevice` 后端；
- 实现纯 Rust SHA-256 模块（`sha256.rs`，约 120 行），通过标准测试向量（空字符串、`"abc"`）验证；
- 实现 Merkle 树（`hash_tree.rs`，约 150 行）的构建与验证逻辑；
- 实现 `DmCryptDevice`（约 350 行）和 `DmVerityDevice`（约 200 行）基础功能；
- 实现 `PassthroughDevice` 和 `DmStackBuilder` 组合框架（约 180 行）。

验证结果：

- `cargo test` 通过全部初始单元测试；
- `demo-crypt` 输出 `ciphertext_differs: true`——验证明文写入后底层设备呈现为密文；
- `demo-verity` 输出 `tamper_detected: true`——验证篡改数据块后完整性校验失败。

阶段意义：此阶段完成了项目的“地基”建设。统一的 `BlockDevice` trait 抽象使得后续所有功能扩展（加密、完整性校验、统计、组合）均可基于同一接口实现，为后续的双目标架构奠定了坚实基础。

5.2.2 6 月 23 日–28 日：用户态工具与文档完善

关键事件：

- 完善 `dmctl` 命令行工具（约 800 行），支持 `init-image`、`crypt-write`、`crypt-read`、`verity-root`、`verity-verify`、`verity-format-dm`、`tamper`、`benchmark` 等子命令（commit `c72462b2`、`2f8a6711`）；
- 编写《设计说明》和《Asterinas 接入指南》初版；
- 补充 AI 工具使用声明（commit `a0dbb0f5`）；

- 添加 LaTeX 技术文档框架和答辩 PPT 初稿（commit `f5878935`、`dabfee4d`）。

阶段成果：用户态工具支持完整的镜像操作、加密/解密、完整性校验、篡改模拟和性能基准测试，为后续内核集成的快速验证提供了可靠的“安全网”。

5.3 第二阶段：Linux 兼容性与内核集成启动（2026-06-29 至 2026-07-14）

5.3.1 6 月 29 日：Linux XTS 兼容性验证

关键事件：

- 编写 `scripts/verify_linux_xts_compat.sh` 交叉验证脚本；
- 完成 Rust 实现与 Linux 内核 `aes-xts-plain64` 的逐字节比对；
- 固定向量 SHA-256 校验和：`4334c17f...`，确认完全一致。

验证结果：脚本输出 `aes-xts-plain64 compatibility: PASS`。

阶段意义：此阶段验证了本项目 AES-XTS 加密引擎与 Linux 内核的完全兼容性，这是后续 `cryptsetup` 工具能够无缝工作的前提。逐字节比对保证了即使是最微小的实现偏差（如 IV 生成的小端序处理、GF(2^{128}) 乘法）也不会破坏互操作性。

5.3.2 7 月 14 日：Asterinas 内核补丁 0001–0004 完成

这是项目从“用户态验证”迈向“内核集成”的关键转折点。

补丁详情：

- **补丁 0001 (feat: dm integration)**：创建 `aster-dm` 组件骨架，实现 crypt/verity target、VirtIO 适配和基础 BIO 路径（commit `be47b160`）。代码量约 1000 行，涉及 `Cargo.toml`、`Components.toml`、`device.rs`、`crypto.rs`、`verity.rs` 等文件；
- **补丁 0002 (fix: validate BIO paths)**：修复 BIO DMA segment 方向访问问题，加入 ext2 超级块魔数自检（`self_test_ext2_magic`）。这是启动早期验证加解密路径正确性的关键安全机制；
- **补丁 0003 (rootfs switch)**：实现 `dm-root` init 模式，通过 `pivot_root` 将系统根切换到受保护 mapper。采用两级引导方案（bind mount → chroot → pivot_root）解决 Asterinas rootfs 特殊性；
- **补丁 0004 (Rust Device Mapper)**：完整实现 `/dev/mapper/control` 的 Linux `dm_ioctl` 兼容接口(约 520 行),支持 `DM_DEV_CREATE`、`DM_TABLE_LOAD`、`DM_DEV_SUSPEND`、`DM_DEV_STATUS`、`DM_DEV_REMOVE` 等命令，使 `cryptsetup` 能够在 Asterinas 中控制加密卷。

验证结果：

- `make kernel` 构建成功；

- R01：`cryptsetup open/read/close` 在 KVM 中 PASS；
- R02：`dm-crypt` 受保护根文件系统启动 PASS，串口输出 `PASS system root switched to protected mapper`；
- R03/V01：`dm-verity` 正常根文件系统启动 PASS，串口输出 `status=verified`；
- C01–C05：正确/错误密钥、4/32/256 KiB 往返全部 PASS；
- V02–V07：单字节篡改后全部被拒绝，输出 `status=integrity-rejected`，无根切换 PASS。

证据：12 组测试用例的原始串口日志已归档至 `docs/evidence/stabilization/correctness-l`

阶段意义：此阶段实现了项目的核心功能目标——dm-crypt 和 dm-verity 在真实 Asterinas 内核中运行，并通过了全部正确性测试。R02 和 R03 证明了受保护根文件系统可以成功启动，V03–V07 证明了单字节物理篡改即可被检测并拒绝。

5.4 第三阶段：性能优化与基准框架（2026-07-28 至 2026-08-03）

5.4.1 7 月 28 日：补丁 0005 – 热路径优化

关键事件：

- **补丁 0005（optimize dm hot path）**：
  - 实现 AES-NI 硬件加速后端（`ostd/arch/x86/cpu/aesni.rs`，约 174 行），在保存/恢复 FPU/XMM 状态下使用 AES-NI 指令，启动日志显示 `backend=AES-NI`；
  - 实现 `CryptScratchPool` 有界临时缓冲池（默认 32），减少热路径动态分配；
  - 实现认证后哈希块缓存：只有沿完整 Merkle 路径验证成功的哈希块才允许插入，缓存命中时跳过上层哈希计算（计数器 `auth_shortcuts` 记录）；
  - 对 4 KiB 对齐 BIO 直接复用已完成数据（`data_rereads=0`）；
  - 增加 `/proc/dm-verity-stats` 和 `/proc/dm-crypto-backend` 可观测接口。
- 编写 `tools/run_dm_crypt_bench.sh` 和 `tools/run_dm_verity_bench.sh` 基准运行脚本。

验证结果：

- 启动日志明确输出 `AES backend=AES-NI`；
- `data_rereads=0` 确认 BIO 复用生效，验证了“对 4KiB 对齐 BIO 直接复用已完成数据”的优化；
- `hash_hits` > 0 和 `auth_shortcuts` > 0 确认缓存生效；
- `/proc/dm-verity-stats` 输出所有关键计数器。

5.4.2 8 月 3 日：补丁 0006 – 性能归因与正式基准合同

关键事件：

- **补丁 0006（performance attribution）**：
  - 实现 `/proc/dm-performance-stats`（约 60 行），提供细粒度阶段计时：dm-crypt 包括 `validation_ns`、`scratch_ns`、`copy_ns`、`crypto_ns`、`lower_wait_ns`；dm-verity 包括 `data_sha256_ns`、`cache_lookup_ns`、`hash_wait_ns`、`hash_sha256_ns`、`digest_compare_ns`、`cache_insert_ns`；
  - 通过启动参数 `dm.perf_stats=1` 启用，stats=0/1 A/B 对照量化观察者开销；
  - 固化正式基准合同（`docs/benchmark_contract.md`），规定五轮、交替顺序、中位数报告等标准。
- 编写 `scripts/run_fio_matrix.sh`（约 500 行）自动化矩阵运行脚本，支持 Linux host 和 Asterinas KVM 双平台；
- 编写 `scripts/run_filebench_matrix.sh`、`scripts/run_sqlite_matrix.sh`、`scripts/run_boo` 等补充测试脚本。

阶段意义：此阶段将性能优化从“经验驱动”升级为“数据驱动”。通过精确的阶段计时，可以量化每个子系统的开销（如 AES-XTS 占 1.82%，SHA-256 占 5.74%），指导后续优化方向。正式基准合同确保了所有性能数据的可复现性和可比性。

5.5 第四阶段：性能矩阵执行与证据归档（2026-08-09 至 2026-08-11）

5.5.1 8 月 9 日：真实性审计与重新执行

关键事件：

- 发现早期 Asterinas dm-crypt FIO 因 mapper 初始化失败将不存在的路径当作普通文件测试，产生数量级异常数据（高达数千 MiB/s），判定为 `HARNESS_BUG`；
- 修复 runner（增加设备存在性检查），重新执行 Asterinas dm-crypt 和 dm-verity FIO，确保使用 guest 内真实 mapper 设备；
- 正式执行 Linux QEMU virtio raw/dm-crypt 13+13 项 FIO，以及 dm-verity 读 7 项。

结果：

- Asterinas dm-crypt：13 项 × 5 轮完成，4K 顺序读中位带宽 8.908 MiB/s；
- Asterinas dm-verity：7 项 × 5 轮完成，6 项写入 N/A，4K 顺序读中位带宽 7.505 MiB/s；
- Linux QEMU virtio raw/dm-crypt：各 13 项 × 5 轮完成，dm-verity 读 7 项；4K 顺序读中位带宽分别为 11.391、10.942、10.704 MiB/s。

阶段意义：此阶段体现了项目对性能数据的诚实态度——宁可重新执行全部测试，也不使用无法确认真实性的“好看数据”。这对维护项目在评审中的可信度至关重要。

5.5.2 8 月 10 日：补充性能测试与证据整理

关键事件：

- Filebench fileserver：raw 5 轮、dm-crypt 5 轮 PASS（dm-verity 因只读与写密集负载不兼容记为 N/A）；
- SQLite 三目标（raw、dm-crypt、dm-verity）各 5 轮 PASS；
- Asterinas 三场景（plain、dm-crypt、dm-verity）各 5 轮启动耗时测试 PASS。

关键数据：

- 启动中位耗时：plain 51.331303 s，dm-crypt 51.629820 s（+0.582%），dm-verity 52.743415 s（+2.751%），验证了“dm-verity 不显著增加 boot 时间”；
- Filebench fileserver 中位 ops/s（Linux QEMU/KVM guest）：raw 27855.155，dm-crypt 25666.016；
- SQLite 10000 行总耗时（Linux QEMU/KVM guest）：raw 0.138095s，dm-crypt 0.143336s，dm-verity 0.001151s（只读查询）。

5.5.3 8 月 11 日：组合栈（补丁 0007）与最终整合

这是项目从“单层保护”迈向“组合保护”的最终形态。

关键事件：

- **补丁 0007（feat: dm stack crypt-verity）**：
  - 支持 `dm_stack=crypt,verity` 启动参数，自动识别堆叠场景；
  - 实现 `scripts/build_stack_rootfs.sh`（约 150 行）构建组合栈镜像：先创建 plaintext ext2 + verity hash tree，再整盘 AES-XTS 加密；
  - 实现 `scripts/run_dm_stack.sh` 启动脚本，支持 Docker 和宿主机两种运行模式；
  - 串口打印 `[dm] stacked mapper: vdc -> dm-crypt0 -> dm-verity0`，明确展示保护链。
- 运行 `final_demo.sh` 完成全部正确性和性能验收；
- 合并所有最终评测代码与测试材料（commit `fda85aa4`、`84fe076e`）。

验证结果：

- 组合栈串口输出包含 `stacked mapper` 标记；
- ext2 文件系统通过两层 mapper 成功挂载；
- 根切换 PASS，输出 `PASS system root switched to protected mapper`；
- `review_release_check.py` 全部通过。

阶段意义：组合栈实现了赛题的最终形态——同时提供机密性（dm-crypt）和完整性（dm-verity）保护。这与用户态 `demo-stack` 的设计完全一致，验证了双目标架构的正确性。

5.6 第五阶段：文档与答辩材料完成（2026-08-11 至 2026-08-14）

关键事件：

- 完善技术文档（`docs/document/main.pdf`，9 章 + 附录，约 120 页），包含设计说明、架构图、测试报告、合规声明和用户手册；与设计赛道文档互为补充；
- 完成设计赛道文档（`docs/design_doc/design_doc.pdf`，10 章，约 80 页），按大赛大纲组织题目分析、计划、进展、分工与收获；
- 制作答辩 PPT（`presentation_project.pptx`），突出双目标架构、安全优化和性能证据；
- 录制演示视频（作品演示视频.mp4），展示运行环境介绍与受保护根文件系统启动全过程；
- 更新 README、证据索引和最终报告。

5.7 关键里程碑时间线

表 5.1: 关键里程碑时间线

日期	里程碑	关键成果
06-23	用户态核心完成	`dm` crate（约 1500 行）、`dmctl` 工具、13+ 单元测试 PASS
06-29	Linux XTS 兼容	逐字节比对 PASS，固定向量 SHA-256 匹配
07-14	内核 rootfs 切换	R01–R03、C01–C05、V01–V07 全部 PASS，12 组测试用例通过
07-28	热路径优化	AES-NI、scratch pool、hash cache、BIO 复用、`/proc/dm-verity-stats`
08-03	性能归因框架	`/proc/dm-performance-stats`（11 个阶段计数器）、五轮基准脚本
08-09	真实性审计	重新执行 Asterinas FIO，修复 runner，使用真实 mapper
08-10	性能矩阵完成	46 PASS 单元格、Filebench 5+5、SQLite 5+5+5、启动 5+5+5
08-11	组合栈完成	补丁 0007、`build_stack_rootfs.sh`、`run_dm_stack.sh`
08-14	文档与答辩准备	技术文档 PDF（120 页）、答辩 PPT、演示视频、设计赛道文档（80 页）

5.8 Git 提交记录概览

以下为主要 Git 提交记录（与 GitLab 仓库历史逐条对应），完整反映了项目的开发节奏和功能演进：

表 5.2: Git 提交记录概览

序号	日期	Commit Hash	提交说明	作者
1	06-23	6bb9bc92	初始提交，仅源码，屏蔽编译产物	潘利源
2	06-23	40a52179	feat: add Linux-compatible device mapper implementation	潘利源
3	06-23	c72462b2	fix: validate Asterinas dm BIO paths	潘利源
4	06-23	2f8a6711	接入受保护根文件系统并完善项目文档	潘利源
5	06-23	a0dbb0f5	补充国产 AI 工具使用说明	潘利源
6	06-29	f5878935	添加 LaTeX 项目文档、PDF 与答辩 PPT	张乐桁
7	06-29	dabfee4d	将答辩 PPT 放到项目根目录并更新 README 链接	张乐桁
8	07-14	5f345a18	完成安全存储性能对照与国赛验收证据	潘利源
9	07-25	e07ad006	修正评审材料并固化 KVM 基准证据	潘利源
10	07-28	c1f9c472	优化 dm 热路径并补齐评审基准流程	潘利源
11	08-03	40d8abba	增强 dmctl 性能测试框架，支持 IO 分解与可配置规模	陈韵
12	08-03	ca5afe46	完善安全存储性能验证与交付材料	潘利源
13	08-09	5717d925	更新 dm 完成队列性能指标与证据	潘利源
14	08-09	21aa566d	更新性能测试数据与优化量化说明	陈韵
15	08-09	7431e5de	统一正式性能口径并补充优化证据	潘利源
16	08-09	0e7f4c77	修正运行时能力声明并补充 KVM 回归验证	潘利源
17	08-09	09f62e59	补充完整测试进度	潘利源
18	08-10	1ac6325c	整理性能测试合同、FIO/Filebench/SQLite/-Boot 脚本与证据目录	张乐桁
19	08-10	fc548a9e	整合最终评测代码与测试材料	潘利源
20	08-11	6385e70b	feat: add kernel Combined crypt→verity stack path	潘利源
21	08-11	fda85aa4	整合最终评测代码与测试材料	潘利源
22	08-11	84fe076e	合并 GitLab 最新代码并同步最终评测材料	潘利源
23	08-11	8b2018e7	完善更新图表	陈韵

此外，星绽内核集成补丁（`asterinas_port/patches/`）包含更细粒度的演进历史：

- **0001**（06-23）：组件集成与目标定义——创建 `aster-dm` 组件骨架
- **0002**（06-23）：BIO 路径验证与自检——EXT2 魔数自检机制
- **0003**（06-24）：根文件系统 `pivot_root` 切换——两级引导方案
- **0004**（07-14）：Runtime 控制面与 `cryptsetup` 兼容——`/dev/mapper/control`

- **0005**（07-28）：性能优化——缓存、缓冲池、AES-NI、基准
- **0006**（08-03）：性能阶段归因与正式合同——`/proc/dm-performance-stats`
- **0007**（08-11）：Combined 叠栈——`dm_stack=crypt,verity`

5.9 各阶段代码量统计

表 5.3: 各阶段代码量统计（约）

阶段	新增代码量	主要模块
用户态核心	约 1500 行 Rust	`block_device`、`dm_crypt`、`dm_verity`、`hash_tree`、`sha256`、`dmctl`
Linux 兼容性	约 1000 行 Rust	`DmCryptTable`、`dm_control.rs`、`runtime.rs`
内核集成	约 2500 行 Rust	补丁 0001–0004，BIO 适配、启动参数、rootfs 切换
性能优化	约 800 行 Rust	AES-NI、scratch pool、hash cache、阶段归因
组合栈	约 150 行 Rust + 约 300 行 Shell	补丁 0007、镜像构建、启动脚本
测试脚本	约 2000 行 Shell/Python	FIO 矩阵、Filebench、SQLite、Boot、证据验证
文档	约 200 页 LaTeX/Markdown	技术文档、设计赛道文档、README、用户手册

5.10 阶段性成果总结

阶段	可运行成果	可复核证据
用户态核心	`dmctl demo-crypt`、`dmctl demo-verity`	单元测试日志、命令行输出
Linux 兼容性	`verify_linux_xts_compat.sh`	固定向量 SHA-256
内核集成	`tools/run_dm_crypt.sh`、`tools/run_dm_verity.sh`	12 组 KVM 正确性日志
性能优化	`scripts/run_fio_matrix.sh`、`/proc/dm-*-stats`	五轮 FIO 原始 JSON/CSV
组合栈	`scripts/run_dm_stack.sh`	组合栈串口日志
文档答辩	技术文档 PDF、PPT、演示视频	全部提交完成

5.11 小结

本章按时间线记录了项目从 2026 年 6 月 23 日至 8 月 14 日的全部重要进展。项目经历了五个关键阶段：

1. **用户态核心实现**（06-23 至 06-28）：完成了约 1500 行 Rust 核心代码和 800 行命令行工具，通过全部单元测试；
2. **Linux 兼容性与内核集成**（06-29 至 07-14）：验证了 AES-XTS 逐字节兼容，完成了补丁 0001–0004，12 组 KVM 正确性测试全部 PASS；
3. **性能优化与基准框架**（07-28 至 08-03）：实现了 AES-NI、scratch pool、hash cache、BIO 复用和阶段归因，建立了五轮基准框架；
4. **性能矩阵与组合栈**（08-09 至 08-11）：完成 46 PASS 性能矩阵，实现补丁 0007 组合栈，启动开销 <3%；
5. **文档与答辩准备**（08-11 至 08-14）：完成技术文档（120 页）、设计赛道文档（80 页）、答辩 PPT 和演示视频。

所有关键里程碑均有 Git 提交记录、补丁文件或证据日志支撑，完整展示了项目的开发节奏和技术演进路径。项目最终成果全面覆盖赛题五项技术指标，代码、证据、文档和演示材料均已按大赛要求提交至指定托管平台。

第 6 章　系统测试情况

本章测试体系速览

测试层级	测试内容	用例数量	结果
单元测试	Rust workspace 核心逻辑验证	13+	全部 PASS
兼容性测试	Linux aes-xts-plain64 逐字节比对	1 组固定向量	PASS
正确性测试	KVM 环境功能验证	12 组（R01–R03、C01–C05、V01–V07）	全部 PASS
性能测试	FIO、Filebench、SQLite、Boot	46 PASS + 6 N/A	全部完成



图 6.1: 四层测试金字塔

6.1 测试体系概述

项目采用四层门禁测试体系，自底向上逐层验证，确保功能正确性、Linux 兼容性和性能可复现性：

1. **用户态单元测试（第 1 层）**：验证加密算法、哈希树、映射器逻辑和错误处理。在用户态快速迭代，不依赖内核环境，作为整个项目的“安全网”；
2. **Linux 兼容性测试（第 2 层）**：与 Linux 内核 `aes-xts-plain64` 逐字节比对，确保密文生成与 Linux 完全一致，这是与 cryptsetup 互操作的前提；
3. **Asterinas KVM 正确性测试（第 3 层）**：在真实 VirtIO BIO 路径下验证 mapper 功能、rootfs 切换和物理损坏检测，覆盖回归、加密、完整性三大类共 12 组用例；
4. **正式性能测试（第 4 层）**：FIO、Filebench、SQLite 和启动耗时的五轮基准，每个有效单元格固定五轮，报告中位数和标准差。

所有测试均要求真实执行并保留原始证据（串口日志、JSON、CSV），只有满足此条件的项才标记为 `PASS`。历史无效数据（如早期假 mapper 测试）已被明确分类并隔离。

6.2 测试环境基线

表 6.1: 正确性测试固定环境

组件	版本/配置
Asterinas commit	`9d79542cadddc6164ad3f31fd8cba776969d85a9`
Docker 镜像	`asterinas/asterinas:0.18.0-20260618`
镜像摘要	`sha256:7510071cb103309cc9396b970325c354713da749dfee34a807108248cc97331e`
Rust 工具链	`nightly-2026-04-03`
架构	x86_64
虚拟化	QEMU/KVM，`-cpu host`，1 vCPU，8 GiB 内存
存储接口	`virtio-blk`，符合赛题标准化要求
Cargo 缓存	持久化 volume（`asterinas-cargo`）
rustup 缓存	持久化 volume（`asterinas-rustup`）

性能测试分 Linux QEMU virtio 和 Asterinas KVM 两个平台分别执行，详细环境见 `PLATFORM_EVAL_ENVIRONMENT.md`。

6.3 单元测试

Rust workspace 单元测试覆盖以下核心场景（13+ 个测试用例）：

表 6.2: 单元测试覆盖场景

测试模块	覆盖场景
`crypt_test`	dm-crypt 往返加密、错误密钥无法恢复明文
`verity_test`	dm-verity 正常读取、篡改检测、错误根哈希拒绝、只读强制
`stack_test`	组合栈安全能力传播、插桩统计
`cryptsetup_compat_test`	Linux table 解析、offset 和 plain64 语义验证
`file_device_test`	文件后端持久化、只读模式拒绝写入
`security_boundary_test`	容量溢出拒绝、并发安全、不同扇区不同密文

执行方式：`cargo test --workspace`

6.4 Linux 兼容性测试

6.4.1 测试目的

确保 AES-256-XTS 实现与 Linux 内核 `aes-xts-plain64` 逐字节兼容，这是 `cryptsetup` 工具能够无缝工作的前提。

6.4.2 测试方法

1. 在 Linux 系统中使用 `dmsetup` 创建 `aes-xts-plain64` 映射器；
2. 使用相同密钥（512 位）和相同明文模式（全 0x5a 的一个扇区）分别通过 Linux 和 Rust 实现加密；
3. 使用 `cmp` 逐字节比较两个密文；
4. 使用 `sha256sum` 记录固定向量校验和。

6.4.3 测试结果

- 固定向量 SHA-256：`4334c17f4b0380b3c1f902509cce0c0f98d9fa77aa93f6407ec940215d77921d`
- 逐字节比对结果：`PASS`
- 证据脚本：`scripts/verify_linux_xts_compat.sh`

6.5 Asterinas KVM 正确性测试

6.5.1 回归测试（R01–R03）

回归测试验证基础构建和关键用户流程的正确性，确保核心功能在每次变更后仍然正常工作。

表 6.3: 回归测试结果

编号	测试内容与验证方法	结果
R01	在 Asterinas KVM 中使用真实 `cryptsetup 2.7.5` 对加密卷执行 `open`、`dd read`、`close` 完整流程，验证控制面与数据面协同工作	PASS
R02	通过 `dm_crypt.*` 启动参数创建加密 mapper，在 initramfs 中挂载并执行 `pivot_root` 切换到受保护根文件系统	PASS
R03	通过 `rootfs_verity.*` 启动参数创建 verity mapper，验证 on-disk 哈希树，执行 `pivot_root` 切换到受保护根文件系统	PASS

证据路径:

- R01: `docs/evidence/stabilization/correctness-kvm/regression/R01-cryptsetup-persistent-cache.log`
- R02: `docs/evidence/stabilization/correctness-kvm/regression/R02-crypt-rootfs-persistent-cache.log`
- R03: `docs/evidence/stabilization/correctness-kvm/regression/R03-verity-rootfs-persistent-cache.log`

6.5.2 dm-crypt 加密测试（C01–C05）

验证 AES-256-XTS 加密的正确性，包括正确密钥、错误密钥和不同 I/O 大小的往返测试。

表 6.4: dm-crypt 功能测试结果

编号	测试内容与验证方法	结果
C01	正确密钥下通过 mapper 写入数据，读回后 SHA-256 与原始明文一致；直接读取底层设备验证为密文	PASS
C02	使用错误密钥读取 C01 写入的密文，读回数据 SHA-256 与正确明文不同，验证明文无法恢复	PASS
C03	4 KiB 块大小写入/读取往返，逐字节比对数据一致性	PASS
C04	32 KiB 块大小写入/读取往返，逐字节比对数据一致性	PASS
C05	256 KiB 块大小写入/读取往返，逐字节比对数据一致性	PASS

证据路径:

- C01–C05: `docs/evidence/stabilization/correctness-kvm/crypt/C01-C05-kvm.log`

6.5.3 dm-verity 完整性测试（V01–V07）

验证 dm-verity 的完整性校验能力，包括正常读取和各类物理损坏场景。V03–V07 的物理篡改位置记录在 `PHYSICAL_MUTATION_MANIFEST.md` 中，每个用例只翻转目标块内偏移 17 的一个字节，且始终使用原始正确根哈希（不重新计算）。

表 6.5: dm-verity 功能测试结果

编号	测试内容与验证方法	结果
V01	未篡改镜像，使用正确根哈希启动，`status=verified`，根切换 PASS	PASS
V02	镜像不变，仅将启动参数中的根哈希替换为错误值（修改一个字节），mapper 创建被拒绝，输出 `status=integrity-rejected`	PASS
V03	物理块 0（首个数据块）偏移 17 处单字节翻转，使用原始正确根哈希，mapper 创建被拒绝	PASS
V04	物理块 32768（中间数据块）偏移 17 处单字节翻转，使用原始正确根哈希，读取返回 I/O 错误	PASS
V05	物理块 65535（最后一个数据块）偏移 17 处单字节翻转，使用原始正确根哈希，读取被拒绝	PASS
V06	物理块 65536（首个叶子哈希块）偏移 17 处单字节翻转，使用原始正确根哈希，mapper 创建被拒绝	PASS
V07	物理块 66048（首个中间哈希块）偏移 17 处单字节翻转，使用原始正确根哈希，mapper 创建被拒绝	PASS

证据路径:

- V01: `docs/evidence/stabilization/correctness-kvm/regression/R03-verity-rootfs-persistent-cache.log`
- V02: `docs/evidence/stabilization/correctness-kvm/corruption/V02-wrong-root.log`
- V03–V07: `docs/evidence/stabilization/correctness-kvm/corruption/V0[3-7]-targeted-probe.log`

6.5.4 物理篡改清单详情

表 6.6: 物理篡改清单（V03–V07）

编号	篡改区域	物理块号	字节偏移	篡改后块 SHA-256
V03	首个数据块	0	17	`b31dc2a37cc0c3c1...`
V04	中间数据块	32768	134217745	`1e12a90f8a205a6a...`
V05	最后一个数据块	65535	268431377	`dbeaf0b712cd66aa...`
V06	首个叶子哈希块	65536	268435473	`5026ae43d7fd0385...`
V07	首个中间哈希块	66048	270532625	`557feb6640edc47d...`

所有篡改均只翻转一个字节，且 ** 继续使用原始正确根哈希 ** (`b1a6dcd4...`)，验证了 dm-verity 的篡改检测能力不依赖于重新计算根哈希。

6.6 性能测试

性能测试遵循 `docs/benchmark_contract.md` 中定义的合同，所有有效单元格固定运行五轮，报告均值、中位数和标准差，不剔除异常值。Linux 与 Asterinas 结果分平台列示，不进行跨平台倍率比较。

6.6.1 FIO 矩阵

FIO 矩阵覆盖 4 KiB、32 KiB、256 KiB 三种块大小，顺序/随机读/写四种操作，QD=1（额外 4 KiB randread QD=16）。

表 6.7: FIO 矩阵完成情况

范围	有效单元格	结果
Asterinas KVM dm-crypt	13 项 × 5 轮	`PASS`
Asterinas KVM dm-verity	7 项 × 5 轮	`PASS`（6 项 N/A）
Linux QEMU virtio raw	13 项 × 5 轮	`PASS`
Linux QEMU virtio dm-crypt	13 项 × 5 轮	`PASS`
Linux QEMU virtio dm-verity 读	7 项 × 5 轮	`PASS`（6 项 N/A）

代表性结果（五轮中位带宽，MiB/s）：

Asterinas 256 MiB 正式合同（4 KiB 同步读、QD1；相对值为配对 raw，不与下表 8.908 / 7.505 混算）：

设备	中位吞吐	相对配对 raw
raw（两组共十轮）	4.249	100.0%
dm-crypt	4.059	95.6%
dm-verity	3.815	87.3%

负载	块大小	QD	Linux raw	Linux dm-crypt
read	4 KiB	1	11.391	10.942
read	256 KiB	1	134.623	129.474
randread	4 KiB	16	58.931	58.947
randwrite	256 KiB	1	130.124	109.453

负载	块大小	QD	Asterinas dm-crypt	Asterinas dm-verity
read	4 KiB	1	8.908	7.505
read	256 KiB	1	97.456	53.363
randread	4 KiB	16	8.614	4.618
randwrite	256 KiB	1	93.541	N/A（只读）

证据路径：Linux QEMU virtio FIO 见 `docs/evidence/linux-qemu-fio-20260816/`；Asterinas FIO 见 `docs/evidence/final-performance-20260811/PERFORMANCE_MATRIX.csv`。

6.6.2 补充性能测试

表 6.8: 补充性能测试结果（五轮中位数）。Filebench 与 SQLite 为 Linux QEMU/KVM guest；启动耗时为 Asterinas。

测试	raw/plain	dm-crypt	dm-verity
Filebench fileserver（ops/s）	27855.155	25666.016	N/A
Filebench fileserver（MB/s）	656.9	605.0	N/A
SQLite 10000 行总耗时（s）	0.138095	0.143336	0.001151（只读查询）
Asterinas 启动耗时（s）	51.331303	51.629820	52.743415

SQLite dm-verity 只执行查询阶段，不能与完整写事务总耗时横比。启动耗时从 QEMU 进程启动计至串口出现受保护根文件系统 PASS 标记。

证据路径：

- Filebench / SQLite（Linux QEMU guest）：`docs/evidence/linux-qemu-filebench-sqlite-2026`
- Boot：`docs/evidence/final-performance-20260811/boot-matrix/`

6.7 数据真实性与审计

6.7.1 历史问题处理

早期 Asterinas dm-crypt FIO 测试中，mapper 设备未正确创建（因启动参数错误或自检失败），但测试脚本继续执行，fio 将不存在的设备路径（如 `/dev/mapper/dm-crypt-bench`）当作普通文件创建并测试，产生了数量级异常的数据（高达数千 MiB/s）。

该批次结果被明确分类为 `HARNESS_BUG`，已从正式矩阵中移除。重新执行的测试使用修复后的 runner（增加设备存在性检查），确保每个数据点均来自 guest 内真实 mapper 设备。

6.7.2 正式数据标准

- 每个有效单元格包含 5 轮原始 JSON/CSV；
- 每轮测试均保留完整的 fio 输出和环境 metadata；
- 报告中位数和标准差，不选择最好一轮；
- 旧数据、失败重试和调试日志不进入最终目录；
- 生成器脚本不读取无效数据目录。

6.8 静态检查与门禁

项目包含自动化静态检查脚本，确保提交材料的完整性和一致性：

```
python3 scripts/check_benchmark_package.py
python3 scripts/verify_kvm_evidence.py
python3 scripts/review_release_check.py
```

检查内容包括：

- 必要证据文件是否存在（kernel build 日志、R01–R03、C01–C05、V02–V07）；
- 最终 FIO 矩阵是否包含 46 个 PASS 单元格；
- 每个 PASS 单元格是否关联 5 轮原始数据；
- 启动矩阵是否包含 plain、dm-crypt、dm-verity 各至少 5 轮；
- 公开文档中是否存在无证据支持的性能夸大表述。

6.9 证据可复核性

所有正式测试证据均以原始形式保存，便于复核：

表 6.9: 证据目录结构

路径	内容
stabilization/build-recovery/	kernel build 日志
stabilization/correctness-kvm/regression/	R01–R03 串口日志
stabilization/correctness-kvm/crypt/	C01–C05 串口日志
stabilization/correctness-kvm/corruption/	V02–V07 串口日志
final-performance-20260811/	最终性能矩阵、报告、原始数据
final-performance-20260811/asterinas-fio/	Asterinas KVM FIO 原始 JSON
linux-qemu-fio-20260816/	Linux QEMU virtio FIO 原始 JSON
linux-qemu-filebench-sqlite-20260817/	Linux QEMU guest Filebench / SQLite
final-performance-20260811/filebench-valid/	Linux host Filebench 归档，不再引用
final-performance-20260811/sqlite/	Linux host SQLite 归档，不再引用
final-performance-20260811/boot-matrix/	启动 15 轮记录
stack/	组合栈启动证据

每条矩阵记录都包含原始证据相对路径，不依赖手工修改数字。完整的复现命令和证据索引见仓库根目录 `README.md` 和 `docs/evidence/README.md`。

6.10 测试门禁汇总

表 6.10: 全部测试门禁状态

测试门禁	结果	证据位置
Rust workspace 单元测试	PASS	`cargo test`
Linux XTS 逐字节兼容	PASS	`verify_linux_xts_compat.sh`
R01–R03 回归测试	PASS	`correctness-kvm/regression/`
C01–C05 加密功能	PASS	`correctness-kvm/crypt/`
V01–V07 完整性校验	PASS	`correctness-kvm/corruption/`
FIO 性能矩阵（46 PASS）	PASS	`final-performance-20260811/`
Filebench（5+5 轮）	PASS	`linux-qemu-filebench-sqlite-20260817/filebench/`
SQLite（5+5+5 轮）	PASS	`linux-qemu-filebench-sqlite-20260817/sqlite/`
Boot 耗时（5+5+5 轮）	PASS	`final-performance-20260811/boot-matrix/`
静态包检查	PASS	`review_release_check.py`

6.11 小结

所有 P0 正确性测试（R01–R03、C01–C05、V01–V07）全部通过，正式性能矩阵包含 46 个有效单元格和 6 个只读 N/A，补充测试（Filebench、SQLite、Boot）均完成五轮验证。

历史无效数据（如早期假 mapper 测试、失败重试）已被明确分类为 `HARNESS_BUG` 并隔离，不进入最终结论。尚未定义 runner 的扩展 P1 项保持 `UNTESTED`，不解释为 PASS，也不解释为产品失败。

完整测试证据已全部归档至 `docs/evidence/` 目录，所有原始日志、JSON、CSV 均可复核。测试结果全面覆盖赛题五项技术指标，为项目的正确性和性能提供了可验证的支撑。

第 7 章　遇到的主要问题和解决方法

本章问题解决速览

序号	问题领域	严重程度	解决方案核心
1	块大小与扇区语义	高	扇区级变换 + checked_add 防溢出
2	BIO 生命周期与错误传播	高	状态机重构 + 错误传播链
3	verity 信任边界与缓存	高	“先认证后插入”原则
4	性能测试真实性	中	设备存在性检查 + 数据分类
5	环境下载与缓存	中	持久化 Docker volumes
6	pivot_root 限制	中	两级引导方案
7	组合栈设备发现	中	堆叠检测逻辑
8	同步原语差异	中	spin::Mutex + 禁用抢占
9	AES-NI FPU 状态保存	中	FpuContext + disable_preempt

7.1　概述

在项目开发过程中，我们遇到了若干涉及操作系统内核语义、密码学兼容性、并发控制和测试可靠性的关键工程问题。这些问题大多源于 Rust 与 C 内核接口的差异、虚拟化环境的特殊性以及安全存储系统的严格性要求。本章按问题领域分类，详细描述每个问题的现象、分析过程和最终解决方案。

7.2　问题解决方法论

在解决上述问题的过程中，我们形成了以下方法论，贯穿整个开发周期：

1. **现象复现优先**：在尝试修复之前，先确保问题可以在受控环境中稳定复现。对于偶发性问题（如 BIO 竞态），通过增加日志和断言将其变为确定性触发；
2. **根因分析驱动**：不满足于“表面修复”，而是追溯到问题的根本原因。例如，BIO 错误传播问题的根因是上层 BIO 的生命周期与下层 BIO 未正确绑定，而非简单的错误码传递；
3. **最小化验证**：每次修复后，只运行与该问题直接相关的最小测试集验证修复效果，避免在整个测试套件中浪费迭代时间；
4. **证据留存**：每个问题的分析过程、中间调试日志和最终修复方案均记录在案，形成可追溯的问题解决链；
5. **回归测试固化**：每个修复都伴随一个新的或增强的测试用例，确保同一问题不会再次出现。

7.3 块大小与扇区语义问题

7.3.1 问题描述

上层文件系统（如 ext2）通常以 4 KiB 块发起 I/O 请求，而 `dm-crypt` 的 `plain64` IV 生成以 512 字节扇区号为单位。Linux 内核的 `dm-crypt` 在处理 4 KiB BIO 时，会将该 BIO 拆分为 8 个 512 字节扇区分别加密，每个扇区使用独立的扇区号作为 IV 的一部分。

在早期实现中，我们将整个 4 KiB 块视为一个整体，使用块号（而非扇区号）作为 IV 输入，导致与 Linux 生成的密文不兼容。此外，`iv_offset` 和 `data_offset` 参数的算术运算存在整数溢出风险。

7.3.2 分析过程

通过 `verify_linux_xts_compat.sh` 脚本对比 Linux 内核和 Rust 实现生成的密文，发现两者不一致。调试发现：

1. Linux 对每个 4 KiB 块内的 8 个扇区使用连续扇区号（block_id $\times$ 8 到 block_id $\times$ $8 + 7$）；
2. `iv_offset` 会累加到每个扇区号上；
3. `data_offset` 以扇区为单位，需要转换为块偏移；
4. 在极端大容量设备场景下，块号与扇区号的乘积可能溢出 64 位整数。

7.3.3 解决方案

在 `DmCryptDevice` 中实现精确的扇区级变换：

```
let sectors_per_block = data.len() / SECTOR_SIZE;
```

```
let sector_delta = block_id * sectors_per_block;
let first_sector = iv_offset + sector_delta;
for (index, sector_data) in data.chunks_exact_mut(SECTOR_SIZE).enumerate() {
    let sector = first_sector + index as u64;
    cipher.encrypt_sector(sector, sector_data)?;
}
```

所有算术运算均使用 `checked_add` 和 `checked_mul` 防止溢出，若溢出则返回 `DmError::OutOfRange`。同时增加了基于 ext2 超级块魔数的启动自检机制（`self_test_ext2_magic`），在启动时验证加解密路径的正确性。

7.3.4 验证结果

修复后，Rust 实现与 Linux 内核输出逐字节一致，固定向量 SHA-256 匹配（`4334c17f...`）。该问题在补丁 0002 中固化。`checked_*` 算术运算在后续所有 I/O 路径中均被采用，防止了类似问题的出现。

7.4 BIO 生命周期与错误传播问题

7.4.1 问题描述

在 Asterinas 内核中，BIO 是异步提交的，上层 mapper 的 BIO 与下层设备的 BIO 之间存在生命周期依赖关系。早期实现中，上层 BIO 在下层 BIO 完成前就被标记为完成，或者在出现错误时未正确恢复上层数据。

具体表现为：

- 下层读失败时，上层仍然返回成功（错误丢失）；
- 下层写成功后，上层 scratch buffer 未正确释放；
- `flush` 操作未同步等待下层完成。

7.4.2 分析过程

通过追踪 `device.rs` 中 `process_crypt` 和 `process_verity` 的执行流程，发现以下问题：

1. 读路径中，下层 BIO 完成后才进行解密，但在错误情况下仍然尝试解密已损坏的数据；
2. 写路径中，加密后的密文保存在 `plaintext_segments` 中，但在提交下层后未正确恢复原始 segment；
3. `flush` 操作直接返回成功，未调用下层设备的 `sync`；
4. 上层 BIO 在下层 BIO 完成回调之前就被 drop，导致 use-after-free 风险。

7.4.3 解决方案

重构 BIO 处理逻辑，严格遵循以下模式：

1. **读路径**：提交下层读 BIO，等待完成后检查状态；若失败则直接完成上层 BIO 并传播错误；若成功则解密数据并写回上层 segment；
2. **写路径**：从 scratch pool 获取缓冲区，加密后通过 `BioSegment::alloc_bytes` 创建新的 bounce segment 提交下层；下层完成后释放 scratch 缓冲区；
3. **Flush**：调用下层设备的 `sync` 并等待完成；
4. **错误传播**：任何下层错误或密码运算错误均导致上层 BIO 以 `BioStatus::IoError` 完成；
5. **生命周期绑定**：使用 `Arc<Mutex<Option<SubmittedBio>>>` 确保上层 BIO 在下层完成回调期间始终存活。

此外，在 `process_crypt` 中增加了逻辑扇区范围验证，确保所有 I/O 不超出 mapper 的逻辑大小。

7.4.4 验证结果

修复后，所有正确性测试（C01–C05、R02）通过，串口日志显示无错误丢失。该问题在补丁 0002 和补丁 0004 中逐步完善，后续所有 BIO 处理代码均遵循相同的生命周期管理模式。

7.5 dm-verity 信任边界与缓存安全

7.5.1 问题描述

dm-verity 的安全性依赖于根哈希作为信任锚，任何数据块或哈希块的篡改都应被检测。然而，引入哈希块缓存后，如果不加限制地缓存哈希块，可能引入以下风险：

1. 未经验证的哈希块被缓存，后续验证使用该缓存绕过实际校验；
2. 缓存的哈希块与数据块之间的信任链断裂；
3. 缓存大小无限制，导致内存耗尽。

7.5.2 分析过程

在设计哈希缓存时，我们面临一个核心矛盾：缓存可以提高性能，但缓存的内容必须可信。如果简单地缓存所有读取过的哈希块，攻击者可能先提交一个伪造的哈希块使其进入缓存，后续所有依赖该哈希块的验证都会错误地通过。

分析 Linux 内核的 `dm-verity` 缓存实现发现，它采用类似的“验证后缓存”策略，但基于 C 语言实现，缺少类型系统层面的安全保证。

7.5.3 解决方案

采用“先认证后插入”策略，并用 Rust 类型系统强制执行：

1. **认证顺序**：验证一个数据块时，从叶子哈希逐级向上验证；每读取一个哈希块，暂时不插入缓存；
2. **完整路径验证**：只有当整个路径（从数据块到根哈希）全部验证通过后，才将本次路径中涉及的所有哈希块插入缓存；
3. **缓存命中捷径**：当某个哈希块已存在于缓存中时，说明该块已经被完整认证过，可以直接使用，无需重新验证其上层哈希（由计数器 `auth_shortcuts` 记录）；
4. **有界缓存**：使用 FIFO 队列，容量由启动参数 `rootfs_verity.cache_blocks` 控制（默认 256），防止内存耗尽。

代码实现位于 `verity.rs` 的 `VerifiedHashBlockCache` 结构体中，Rust 的所有权系统保证了缓存块在被插入前必须经过完整的验证路径，类型系统杜绝了“未验证即插入”的编码错误。

7.5.4 验证结果

V03–V07 的单字节篡改测试均被拒绝，验证了缓存未破坏信任边界。同时，`auth_shortcuts` 计数器在第二轮顺序读中显著增加（从第一轮的 0 增加到第二轮的 200+），证明缓存命中路径确实跳过了上层哈希计算。`/proc/dm-verity-stats` 提供了缓存命中的精确量化数据。

7.6 性能测试真实性问题

7.6.1 问题描述

早期 Asterinas dm-crypt FIO 测试中，mapper 设备未正确创建（因启动参数错误或自检失败），但测试脚本继续执行。fio 将不存在的设备路径（如 `/dev/mapper/dm-crypt-bench`）当作普通文件创建并测试，产生了数量级异常的性能数据（高达数千 MiB/s）。

这些异常数据与真实 mapper 测试结果混在同一证据目录中，构成严重的数据污染。

7.6.2 分析过程

排查测试日志发现：

1. 测试脚本未检查 mapper 设备是否存在；
2. 即使 mapper 创建失败，fio 仍然执行，并自动创建普通文件；

3. 这些“假 mapper”测试的吞吐量远高于真实块设备（真实设备约 10-100 MiB/s，假测试高达 3000+ MiB/s），明显异常；
4. 旧批次中 raw 与 dm-crypt 结果混在一起，无法直接复用。

7.6.3 解决方案

1. **修复 runner**：在 `run_dm_crypt_bench.sh` 和 `run_dm_verity_bench.sh` 中增加设备存在性检查，若 mapper 不存在则立即失败；
2. **数据分类**：将旧批次中无法确认 mapper 真实性的数据标记为 `HARNESS_BUG`，不纳入正式矩阵；
3. **重新执行**：使用修复后的 runner 重新执行所有 Asterinas dm-crypt 和 dm-verity 测试；
4. **证据隔离**：正式证据目录只包含重新执行后的原始 JSON 和日志，旧数据及失败重试不进入最终目录。

7.6.4 验证结果

重新执行后的 Asterinas dm-crypt 和 dm-verity 数据均来自真实 mapper 设备，五轮带宽稳定（4K 顺序读 8.908 / 7.505 MiB/s），正式矩阵仅包含这些已验证的数据。该问题在 `PERFORMANCE_VALIDATION_REPORT.md` 中有明确记录，体现了对数据真实性的零容忍态度。

7.7 环境下载与持久化缓存问题

7.7.1 问题描述

在 Docker 容器中构建 Asterinas 内核时，每次启动新容器都需要重新下载 Cargo 依赖和 rustup 目标，导致构建时间过长（约 15–20 分钟），且网络不稳定时构建失败。

在 R01 测试中，`cryptsetup` 工具需要从 Nix 缓存下载，若网络受限则测试被阻塞。

7.7.2 分析过程

观察发现：

1. Cargo 依赖（`static.crates.io`）偶尔 DNS 解析失败；
2. rustup 目标（如 `x86_64-unknown-none`）每次容器启动都需要重新安装；
3. Nix 构建 `cryptsetup` 时访问 binary cache，在完全离线环境中失败。

7.7.3 解决方案

1. **持久化 Cargo volume**：使用 Docker volume `asterinas-cargo` 保存 `/root/.cargo`，避免重复下载；
2. **持久化 rustup volume**：使用 `asterinas-rustup` 保存 `/root/.rustup`，避免重复安装目标；
3. **固定镜像**：使用固定 digest 的官方开发镜像（`sha256:7510071cb1...`），确保工具链版本一致；
4. **预置 cryptsetup**：在 initramfs 构建阶段包含 `cryptsetup` 二进制，避免运行时下载。

7.7.4 验证结果

修复后，内核构建时间从 15+ 分钟降至 2 分钟，R01 测试不再依赖网络。完整环境验证记录在 `PLATFORM_EVAL_ENVIRONMENT.md` 中。

7.8 pivot_root 限制与引导问题

7.8.1 问题描述

Asterinas 内核的 `pivot_root` 实现要求当前根文件系统（initramfs）不是特殊的“rootfs” 挂载，否则会拒绝切换。在早期实现中，initramfs 的根是内核自动挂载的 rootfs，没有父挂载点，导致 `pivot_root` 返回错误。

7.8.2 分析过程

查阅 Asterinas 内核源码发现，`pivot_root` 要求 old_root 和 new_root 都有有效的父挂载点。直接在内核自动挂载的 rootfs 上调用 `pivot_root` 会失败，因为 rootfs 本身没有父挂载点。这是 Asterinas 与 Linux 在根文件系统挂载语义上的一个重要差异。

7.8.3 解决方案

采用两级引导方案：

1. **第一阶段**：initramfs 脚本通过 `mount --bind / /bootstrap-root` 创建一个 bind mount，将当前根绑定到 `/bootstrap-root`；
2. **第二阶段**：通过 `chroot /bootstrap-root` 重新执行 init 脚本，此时 `pivot_root` 的 old_root 和 new_root 都有有效父挂载点；

3. pivot_root 成功将新根（mapper）切换到 /，旧根保留在 /.initramfs 下供后续使用。

该方案在补丁 0003（rootfs-switch.patch）中实现，并通过 dm-root-stage2 参数区分两个阶段。

7.8.4 验证结果

R02 (dm-crypt) 和 R03/V01 (dm-verity) 均成功执行 pivot_root，串口输出 PASS system root switched to protected mapper，且 initramfs 运行时通过 /.initramfs 路径仍然可访问。

7.9 组合栈的命名与设备发现问题

7.9.1 问题描述

在实现 dm_stack=crypt,verity 组合栈时，verity 的 rootfs_verity.device 参数需要指向 crypt mapper 的名称（如 dm-crypt0）。但早期实现中，verity 组件只接受物理设备名，无法识别已注册的 mapper。

7.9.2 分析过程

在补丁 0007 实现前，aster-dm 的 verity 初始化逻辑直接调用 find_device(device_name)，该函数只查找物理块设备（如 vdc），不会查找已注册的 mapper 设备。这是因为物理设备和 mapper 设备注册在不同的命名空间中。

7.9.3 解决方案

在 lib.rs 的 init 函数中增加堆叠检测逻辑：

```
let stack_crypt_verity = is_crypt_verity_stack(&crypt_mapper_name);
// ...
if stack_crypt_verity && CRYPT_MAPPER.get().is_some() {
    let device_name = crypt_mapper_name.as_str(); // 指向 crypt mapper
} else {
    let device_name = configured_device.as_str();
}
```

同时，当检测到堆叠时，打印清晰的启动日志：

```
[dm] stacked mapper: vdc -> dm-crypt0 -> dm-verity0
```

7.9.4 验证结果

组合栈启动脚本 scripts/run_dm_stack.sh 成功执行，串口日志包含堆叠链和最终的根切换 PASS。证据已归档于 docs/evidence/stack/。

7.10 内核态与用户态同步原语差异

7.10.1 问题描述

用户态代码使用 std::sync::Mutex 进行同步，这是基于操作系统调度器的睡眠锁。但在内核态中，不能在持有锁时睡眠（可能导致死锁或调度延迟），也不能在中断上下文中使用睡眠锁。

早期移植时，部分代码直接使用了与用户态相同的锁策略，导致在 BIO 完成回调中产生死锁或自旋等待过长。

7.10.2 分析过程

通过在内核中添加锁检测断言，发现以下问题：

1. CryptScratchPool 在 BIO 完成回调中被访问，但使用 std::sync::Mutex 会在锁竞争时睡眠；
2. VerifiedHashBlockCache 在 verity 验证路径中被频繁访问，锁持有时间过长；
3. 在中断禁用区域中尝试获取睡眠锁，导致内核 panic。

7.10.3 解决方案

1. 将所有内核态共享状态（CryptScratchPool、VerifiedHashBlockCache）的锁替换为 spin::Mutex（自旋锁）；
2. 在 AES-NI 变换期间通过 disable_preempt 和 irq::disable_local 确保不被中断上下文破坏；
3. 使用 core::sync::atomic::AtomicU64 替代锁保护的热路径计数器(如 stats.hash_hits)，避免在统计路径上加锁；
4. 将缓存插入操作移到路径验证完成后执行，减少锁持有时间。

7.10.4 验证结果

修复后，所有 KVM 正确性测试通过，无死锁或 panic 发生。正式性能测试中，锁竞争开销被降至可接受范围。

7.11 AES-NI 的 FPU/XMM 状态保存问题

7.11.1 问题描述

AES-NI 指令使用 XMM 寄存器进行 SIMD 加密操作。在内核中，XMM 寄存器是 FPU（浮点单元）上下文的一部分，其状态在进程切换时会被保存和恢复。然而，如果在中断处理或内核线程中直接使用 AES-NI 指令而未保存 FPU 状态，可能破坏用户态的 FPU 上下文，导致难以追踪的数据损坏。

7.11.2 分析过程

通过在内核 AES-NI 路径前后添加 FPU 状态检查，发现早期实现未保存 FPU 上下文：

1. 加密操作在 BIO 处理路径中执行，该路径可能在任意进程上下文中被调用；
2. 未保存 FPU 状态意味着当前进程的 XMM 寄存器值可能被 AES 操作覆盖；
3. 这种损坏在用户态程序使用 SIMD 指令时才会显现，难以定位。

7.11.3 解决方案

在 `ostd/arch/x86/cpu/aesni.rs` 中实现 FPU 状态保护机制：

```
// AES-NI 变换前
let mut fpu_context = FpuContext::new();
let _preempt_guard = disable_preempt();
let _irq_guard = irq::disable_local();
fpu_context.save();

// 执行 AES-NI 加密/解密...
encrypt_block_256(block, round_keys);

// AES-NI 变换后
fpu_context.load();
```

`FpuContext::save()` 保存当前 XMM 寄存器状态，`FpuContext::load()` 在操作完成后恢复。`disable_preempt` 和 `irq::disable_local` 确保在 FPU 状态保存和恢复期间不会被调度出去或中断。

7.11.4 验证结果

修复后，AES-NI 启动日志显示 `backend=AES-NI`，所有 KVM 正确性测试通过，未再出现 FPU 状态损坏相关的问题。正式五轮性能测试确认 AES-NI 加速效果稳定可测。

7.12 问题解决效果统计

表 7.1: 各问题修复效果统计

问题领域	修复补丁	影响范围	修复效果
块大小与扇区语义	0002	全部 dm-crypt I/O	Linux XTS 兼容性 PASS
BIO 生命周期	0002, 0004	全部 BIO 处理	12 组正确性测试 PASS
verity 信任边界	0005	dm-verity 读路径	缓存命中不削弱安全性
性能测试真实性	N/A	全部性能证据	正式矩阵仅含真实数据
环境下载	N/A	构建与 R01	构建时间 15min → 2min
pivot_root 限制	0003	rootfs 切换	R02, R03 PASS
组合栈设备发现	0007	组合栈启动	堆叠链打印 + root 切换 PASS
同步原语差异	0005	全部内核并发路径	无死锁/panic
AES-NI FPU 状态	0005	AES-NI 加速路径	启动日志显示 AES-NI

7.13 小结

本章详细记录了项目开发过程中遇到的九个关键工程问题，包括块大小与扇区语义、BIO 生命周期、verity 信任边界、性能测试真实性、环境下载、`pivot_root` 限制、组合栈设备发现、同步原语差异和 AES-NI FPU 状态保护。

每个问题都通过细致的分析、针对性的代码修改和充分的测试验证得到了解决。这些问题及其解决方案不仅确保了项目的正确性，也为后续的 Rust OS 存储安全开发提供了宝贵的经验。关键统计数据：

- 共解决 9 个关键工程问题；
- 涉及补丁 0001–0007 中的 6 个补丁；
- 影响 12 组 KVM 正确性测试和 46 个性能单元格；

- 修复后全部测试 PASS，无已知严重问题遗留。

这些问题的解决过程充分展示了 Rust 在系统编程中“安全与性能兼得”的潜力，以及类型系统和所有权机制在构建可信存储系统时的独特价值。

第 8 章　分工和协作

本章团队协作速览

成员	角色定位	核心贡献领域	关键产出
潘利源	核心架构师	加密/完整性实现、内核集成、性能优化	7 个补丁、约 5000 行 Rust 代码
陈韵	测试工程师	性能测试框架、数据验证、文档完善	测试脚本、6 个 Git commits
张乐桁	文档工程师	测试合同、基准脚本、文档排版、PPT	技术文档 120 页、答辩 PPT

8.1　队员信息

姓名	学校	队伍角色	主要职责
潘利源	华南师范大学	核心开发	核心加解密/完整性实现、星绽内核集成、性能优化、补丁制作
陈韵(jqtxzs)	华南师范大学	测试与文档	性能测试框架增强、测试数据收集与验证、文档完善
张乐桁(lehengzhang)	华南师范大学	测试与文档	性能测试合同编写、FIO/-Filebench/SQLite/Boot 脚本及证据目录整理、技术文档排版、答辩 PPT

8.2 分工详情

8.2.1 潘利源（核心架构师）

核心加密与完整性实现（约 2000 行 Rust）：

- 实现 `BlockDevice` trait 及 `MemoryBlockDevice`、`FileBlockDevice` 后端；
- 实现纯 Rust SHA-256（`sha256.rs`，约 120 行）和 Merkle 哈希树（`hash_tree.rs`，约 150 行）；
- 实现 AES-256-XTS 加密引擎及 Linux `aes-xts-plain64` 兼容性（`crypto.rs`、`dm_crypt.rs`，约 350 行）；
- 实现 `dm-verity` 完整校验逻辑（`dm_verity.rs`，约 200 行）；
- 实现 `DmStackBuilder` 可组合映射器链（`device_mapper.rs`，约 180 行）。

星绽内核集成（约 2500 行 Rust 内核代码）：

- 制作并维护 `asterinas_port/patches/0001--0007` 全部 7 个补丁；
- 实现 BIO 路径适配（`device.rs`）、启动参数解析（`lib.rs`）、根文件系统切换（`pivot_root`）；
- 实现 `/dev/mapper/control` 控制节点（`dm_control.rs`，约 520 行）和运行时 mapper 管理（`runtime.rs`，约 260 行）；
- 实现组合栈（`dm_stack=crypt,verity`，补丁 0007）。

性能优化（约 800 行 Rust）：

- 实现 AES-NI 硬件加速（`ostd/aesni.rs`，约 174 行）及 FPU/XMM 状态保护；
- 实现有界临时缓冲池（scratch pool）减少动态分配；
- 实现认证后哈希缓存和 BIO 数据复用（`verity.rs`）；
- 实现内核性能统计接口（`/proc/dm-performance-stats`、`/proc/dm-verity-stats`）。

核心产出：约 5000 行 Rust 代码、7 个内核补丁、17+ 个 Git commits。

代表性提交：`40a52179`（初始 dm 实现）、`6385e70b`（组合栈）、`e07ad006`（KVM 基准证据）。

8.2.2 陈韵（jqtxzs，测试工程师）

性能测试框架增强（约 400 行 Rust/Shell）：

- 增强 `dmctl` 性能测试框架，支持 I/O 拆分解与可配置规模（commit `40d8abba`）；
- 更新性能测试数据与优化量化说明（commit `21aa566d`）；
- 统一正式性能口径并补充优化证据（协助）。

测试数据与文档完善：

- 完善安全存储性能验证与交付材料（协助 commit `ca5afe46`）；
- 修改完善文档（commits `7350e470`、`dde97a2f`）；
- 完善更新图表（commit `8b2018e7`）。

核心产出：6 个 Git commits、性能测试框架增强、数据验证支持。
代表性提交：`40d8abba`（性能框架增强）、`21aa566d`（性能数据更新）。

8.2.3 张乐桁（lehengzhang，文档工程师）

性能测试合同与基准脚本（约 1000 行 Shell/Python）：

- 编写 `docs/benchmark_contract.md` 定义正式性能测试合同（包含环境、FIO 参数、元数据等完整规范）；
- 整理 FIO、Filebench、SQLite、Boot 测试脚本及证据目录（commit `1ac6325c`）；
- 维护 `scripts/run_fio_matrix.sh`、`scripts/run_filebench_matrix.sh`、`scripts/run_sqlite_` `scripts/run_boot_matrix.sh` 等核心基准脚本。

技术文档与答辩材料（约 200 页 LaTeX/Markdown）：

- 添加 LaTeX 技术文档、PDF 与答辩 PPT（commit `f5878935`）；
- 将答辩 PPT 放到项目根目录并更新 README 链接（commit `dabfee4d`）；
- 在 README 开头添加演示视频百度网盘链接（commit `c22b8b47`）。

核心产出：4 个 Git commits、技术文档 120 页、设计赛道文档 80 页、答辩 PPT。
代表性提交：`f5878935`（LaTeX 文档框架）、`1ac6325c`（性能测试合同与脚本）。

8.3 协作方式

8.3.1 版本管理

- 使用 **GitLab** 作为代码托管平台，所有代码提交均通过 Git 进行版本管理；
- 采用主分支开发模式，重要功能通过补丁序列（`asterinas_port/patches/`）独立演进，便于审计和回溯；
- 提交信息遵循约定格式（`feat:`、`fix:`、`docs:` 前缀），关键提交包含功能描述和关联问题；
- 累计提交 28 次，覆盖从初始框架到最终交付的完整历程。

8.3.2 沟通与同步

- 每周通过线上会议（腾讯会议/钉钉）同步开发进度，每次会议约 1 小时，讨论架构决策、问题排查和下周计划；
- 使用即时通讯工具（QQ/微信）进行日常技术交流和问题排查，平均每天有 3-5 条技术消息交流；
- 重要技术决策（如 BIO 生命周期设计、缓存安全策略）在会议中集体讨论后实施，形成共识文档；
- 关键问题（如性能测试真实性问题、`pivot_root` 限制）通过紧急会议快速解决。

8.3.3 代码审查

- 核心模块（加密、校验、BIO 处理）的代码变更须至少一名其他成员审查后方可合并；
- 补丁提交前，在本地进行完整的 `cargo test` 和 KVM 正确性验证；
- 性能相关的脚本和配置变更需通过 `check_benchmark_package.py` 静态检查；
- 文档变更（README、技术文档）在合并前进行交叉审阅，确保表述准确、无歧义。

8.3.4 决策机制

项目采用“共识驱动”的决策模式，关键决策均经过充分讨论：

- **技术方案决策**：由提出者阐述方案（含优缺点分析），全体成员讨论后投票表决；
- **架构方向决策**：需形成书面决策记录（如补丁设计说明），确保决策可追溯；
- **紧急问题决策**：由核心开发者（潘利源）快速决策并实施修复，事后在周会上说明。

8.3.5 关键决策记录

表 8.1: 关键技术决策记录

时间	决策事项	决策结果	决策依据
06-23	`BlockDevice` trait 设计	采用同步接口（内核用 BIO 封装）	统一抽象、简化用户态测试
07-01	AES-XTS 实现方式	独立实现而非依赖 RustCrypto xts-mode	减少依赖、便于逐字节调优
07-10	补丁组织方式	7 个独立补丁渐进式集成	可审计、可回退、每步可验证
07-15	性能优化优先级	AES-NI > hash cache > scratch pool	硬件加速收益最大、风险最低
08-05	数据真实性处理	废弃旧批次、重新执行	数据诚信优先于“好看”
08-10	文档语言	中文（部分术语保留英文）	便于评委阅读、关键术语准确

8.4 AI 工具辅助协作

项目开发过程中合理使用了 AI 辅助工具（详见 `docs/ai_tool_disclosure.md`），具体使用场景包括：

工具名称	相关大模型	使用场景
WorkBuddy	腾讯混元 Hy3、DeepSeek	阅读和梳理 Asterinas/dm-crypt/dm-verity 代码，辅助分析 BIO 数据路径与构建错误，生成/修改 Rust/Shell 测试和构建脚本，协助整理文档表述，辅助执行测试和 QEMU 验证
通义千问（Qwen）	Qwen-Max	技术文档润色和中英文专业术语校对
豆包（Doubao）	豆包大模型	答辩表述优化与常见问题整理

所有 AI 辅助生成的内容均经过团队成员人工审阅，并通过单元测试、Linux 兼容性验证和 KVM 端到端测试确认正确性。最终技术方案、代码取舍和参赛责任由团队全体成员承担。

AI 工具使用原则：

1. AI 仅作为辅助工具，不替代团队成员的独立思考和判断；
2. 所有 AI 生成的代码必须经过人工审查和测试验证；
3. 关键设计决策（架构、安全策略、性能优化方向）由团队人工讨论决定；
4. AI 交互记录保留在项目 Codex 开发线程中，便于审计。

8.5 团队贡献量化

表 8.2: 各成员贡献量化

成员	Git 提交数	核心代码量	主要贡献领域
潘利源	17+	约 5000 行 Rust	核心加密/完整性(100%)、内核集成(100%)、性能优化(80%)、补丁制作（100%）
陈韵	6+	约 400 行 Rust/Shell	性能测试框架增强（70%）、测试数据收集（50%）、文档完善(40%)
张乐桁	4+	约 1000 行 Shell/Python + 200 页文档	性能测试合同与脚本(60%)、技术文档排版（80%）、答辩 PPT(80%)、证据目录整理（50%）

说明：百分比为相对工作量估计，实际协作中存在交叉支援，例如：

- 性能优化中的缓存设计与验证由潘利源主导编码，陈韵协助测试数据验证；
- 文档撰写由张乐桁主导排版和结构，陈韵和潘利源提供技术内容审阅和补充；
- 性能测试脚本由张乐桁编写框架，陈韵增强具体功能，潘利源验证内核侧数据。

8.6 开发节奏与协作模式亮点

特征	具体实践	效果
定期同步	每周线上会议 + 日常即时通讯	信息同步及时，避免“信息孤岛”
代码审查	核心模块交叉审查 + 补丁验证	代码质量高，关键问题早发现
互补分工	算法/内核/测试/文档各有所长	效率高，无重复劳动
共识决策	重要决策集体讨论 + 书面记录	决策质量高，可追溯
工具辅助	AI 辅助 + 自动化测试脚本	开发效率提升，人工审阅仍把关
文档同步	代码改、文档跟，保持同步	文档与代码一致，避免“写文档浪费时间”

8.7 团队协作感悟

8.7.1 潘利源

“作为核心开发者，我最大的体会是‘代码写得好不如团队协作得好’。最难的 bug（BIO 生命周期问题）不是我一个人解决的——陈韵帮我重现了稳定触发条件，张乐桁帮我梳理了调用栈日志。这次经历让我深刻理解了代码审查的价值。”

8.7.2 陈韵

“我负责测试和文档，一开始觉得‘不如写代码重要’。但当我发现性能测试的假 mapper 问题、帮团队避免了一次数据诚信危机时，我才意识到测试的价值——它是代码质量的最后一道防线。文档也是，没有好的文档，再好的代码评委也看不到价值。”

8.7.3 张乐桁

“作为文档负责人，我需要把团队成员分散的代码、测试结果、设计思路整合成结构化的文档。这个过程让我学会了从‘用户视角’看项目——评委不需要知道每一行代码的细节，但需要知道‘为什么这样做’和‘这样做对不对’。LaTeX 排版 120 页文档的经历也让我对文档工具链有了质的提升。”

8.8 小结

项目团队由三名成员组成，分工明确且互补：潘利源负责核心技术实现和内核集成（约 5000 行 Rust 代码），陈韵负责性能测试框架增强和数据验证（约 400 行代码），张乐桁负责测试合同、脚本、文档（约 1000 行脚本 + 200 页文档）和答辩材料。

协作方式包括：Git 版本管理、每周线上会议、代码审查、共识决策和 AI 辅助。团队累计完成 28 次 Git 提交、7 个内核补丁、12 组 KVM 正确性测试和 46 个性能单元格。

所有成员对最终作品均做出实质性贡献，协作氛围良好，无重大分歧或冲突。团队通过互补分工和高效沟通，按时完成了全部 P0 功能目标、性能基准测试和文档交付。

第 9 章　提交仓库目录和文件描述

本章目录结构速览

一级目录	内容说明	关键文件数
crates/	平台无关的 Device Mapper 核心库和命令行工具	约 15 个 Rust 源文件
crates/dm/	dm crate：BlockDevice trait、dm-crypt、dm-verity	约 3000 行 Rust
crates/dmctl/	dmctl 命令行工具	约 800 行 Rust
asterinas_port/	Asterinas 内核移植补丁和说明	7 个补丁 + README
scripts/	构建、测试、性能和验收脚本	约 30 个脚本文件
docs/	全部文档、证据和设计材料	约 50 个文件
docs/evidence/	正式测试证据	约 100+ 个日志/JSON/CSV 文件
docs/document/	技术实现文档 LaTeX 源码	9 章 + 附录 + PDF
docs/design_doc/	设计赛道文档 LaTeX 源码	10 章 + PDF

9.1 文件统计概览

表 9.1: 关键文件统计

类型	数量/规模	说明
Rust 源代码(用户态)	约 3000 行	`crates/dm/` 核心库
Rust 源代码(内核态)	约 4000 行	补丁 0001–0007 新增内核代码
命令行工具	约 800 行	`crates/dmctl/src/main.rs`
Shell/Python 脚本	约 30 个文件, 约 2000 行	`scripts/` 目录
LaTeX 文档	约 200 页	技术文档 + 设计赛道文档，互为补充
证据文件	约 100+ 个	日志、JSON、CSV、图片
Git 提交	28 次	涵盖完整开发周期
内核补丁	7 个	`asterinas_port/patches/`

9.2 仓库顶层结构

目录/文件	描述
`README.md`	项目概览、快速验证指南、功能命令、性能结果和验收说明；含演示视频网盘链接
`LICENSE`	MIT 开源许可证
`Cargo.toml`	Rust 工作区配置，包含 `dm` 和 `dmctl` 两个 crate，resolver = ”2”
`KVM_CORRECTNESS_MATRIX.md`	KVM 正确性测试完整矩阵（R01–R03、C01–C05、V01–V07）状态表
`KVM_CORRECTNESS_REPORT.md`	KVM 正确性最终报告，含结论、固定基线和证据路径
`PHYSICAL_MUTATION_MANIFEST.md`	dm-verity 物理篡改清单（V03–V07 的块号、字节偏移和 SHA-256）
`PLATFORM_EVAL_ENVIRONMENT.md`	平台评测环境固定基线（Docker image、commit、工具链、验证结果）
`crates/`	平台无关的 Device Mapper 核心库和命令行工具
`crates/dm/`	`dm` crate 源码（约 15 个文件，约 3000 行 Rust）

目录/文件	描述
crates/dm/Cargo.toml	dm crate 的依赖配置（aes、spin 等）
crates/dm/src/block_device.rs	BlockDevice trait 及 MemoryBlockDevice、FileBlockDevice 实现（约 400 行）
crates/dm/src/crypto.rs	AES-256-XTS 加密引擎及 DemoStreamCipher（约 350 行）
crates/dm/src/device_mapper.rs	DmStackBuilder、DmStack、PassthroughDevice（约 180 行）
crates/dm/src/dm_crypt.rs	DmCryptDevice 和 DmCryptTable 解析器（约 350 行）
crates/dm/src/dm_verity.rs	DmVerityDevice 完整性校验实现（约 200 行）
crates/dm/src/error.rs	DmError 错误枚举（约 80 行）
crates/dm/src/hash_tree.rs	Merkle 哈希树构建与验证（约 150 行）
crates/dm/src/lib.rs	模块导出
crates/dm/src/sha256.rs	纯 Rust SHA-256 实现（约 120 行）
crates/dm/src/statistics.rs	I/O 统计计数器（IoStats、InstrumentedDevice，约 150 行）
crates/dm/tests/	单元测试：crypt、verity、stack、兼容性、文件设备、安全边界（约 6 个文件）
crates/dmctl/	dmctl 命令行工具源码
crates/dmctl/Cargo.toml	dmctl 依赖配置
crates/dmctl/src/main.rs	所有子命令实现：init-image、crypt-write、crypt-read、raw-read-hex、crypt-table-fill、verity-root、verity-verify、verity-format-dm、tamper、demo-crypt、demo-verity、demo-stack、benchmark、benchmark-table（约 800 行）
asterinas_port/	Asterinas 内核移植说明与补丁

目录/文件	描述
asterinas_port /README.md	移植说明、补丁应用方式（git am 命令）、关键实现文件路径
asterinas_port /patches/	7 个 Git 补丁（0001–0007），按顺序应用
asterinas_port /patches/0001-*.patch	组件集成与目标定义：创建 aster-dm 组件骨架，实现 crypt/verity target、VirtIO 适配和 BIO 路径
asterinas_port /patches/0002-*.patch	BIO 路径验证与 ext2 魔数自检：修复 DMA segment 方向访问，增加 self_test_ext2_magic
asterinas_port /patches/0003-*.patch	根文件系统 pivot_root 切换：实现 dm-root init 模式，两级引导方案
asterinas_port /patches/0004-*.patch	Runtime 控制面与 cryptsetup 兼容：/dev/mapper/control 和 dm_ioctl 实现
asterinas_port /patches/0005-*.patch	热路径优化：AES-NI、scratch pool、hash cache、BIO 复用、/proc/dm-verity-stats
asterinas_port /patches/0006-*.patch	性能阶段归因与正式合同：/proc/dm-performance-stats、dm.perf_stats=1
asterinas_port /patches/0007-*.patch	dm_stack=crypt,verity 组合栈：堆叠检测、启动日志、build_stack_rootfs.sh

9.3 脚本分类说明

scripts/ 目录包含约 30 个脚本，按功能分为以下五类：

构建类脚本（4 个）

脚本	用途
build_crypt_rootfs.sh	构建 dm-crypt 加密根文件系统镜像（target/crypt-rootfs.img）

脚本	用途
build_verity_rootfs.sh	构建 dm-verity 校验根文件系统镜像（target/verity-rootfs.img）
build_stack_rootfs.sh	构建组合栈（crypt + verity）根文件系统镜像（target/stack-rootfs.img）
build_asterinas_boot_images.sh	构建 plain/dm-crypt/dm-verity 三种启动 ISO（用于启动耗时矩阵）

测试类脚本（6 个）

脚本	用途
check_benchmark_package.py	验证性能包完整性和诚实性（静态检查）
check_filebench.sh	探测 Filebench 可用性
final_demo.sh	一键最终演示（含 --live-kvm 选项）
review_release_check.py	评审版本静态检查（证据、矩阵、文档）
verify_kvm_evidence.py	验证 KVM 正确性证据日志
tests/test_release_tools.py	发布工具的单元测试

性能测试脚本（8 个）

脚本	用途
benchmark.sh	用户态微基准测试（plain/AES-XTS/verity CSV）
benchmark_boot_time.py	启动耗时测量脚本（Python）
run_fio_matrix.sh	正式 FIO 矩阵驱动（Linux host + Asterinas KVM）
run_filebench_dm.sh	单轮 Filebench 测试
run_filebench_matrix.sh	Filebench 5+5 轮矩阵（raw/dm-crypt）
run_sqlite_workload.sh	单轮 SQLite 固定 workload
run_sqlite_matrix.sh	SQLite 5+5+5 轮矩阵

脚本	用途
run_boot_matrix.sh	启动耗时矩阵（plain/dm-crypt/dm-verity 各 5 轮）

验证类脚本（6 个）

脚本	用途
verify_linux_xts_compat.sh	实时与 Linux 内核 dm-crypt 逐字节比对
verify_archived_linux_xts.sh	验证归档的 Linux XTS 固定向量
linux_cryptsetup_baseline.sh	Linux cryptsetup 基准创建
run_asterinas_boot_once.sh	单次 Asterinas 启动（用于 boot 矩阵）
run_dm_stack.sh	运行组合栈（crypt→verity）启动
trace_asterinas_syscalls.gdb	GDB 脚本（追踪 Asterinas syscall）

汇总类脚本（4 个）

脚本	用途
summarize_asterinas_dm_matrix.py	解析 Asterinas fio 串口日志生成 CSV
summarize_filebench_results.py	从 Filebench 日志生成 CSV
summarize_fio_matrix.py	汇总 FIO 矩阵为 summary CSV
workloads/fileserver.f	Filebench fileserver workload 配置文件

9.4 文档目录说明

docs/ 目录包含全部文档、证据和设计材料：

路径	描述
ai_tool_disclosure.md	AI 工具使用声明（工具名称、模型、使用场景、验证机制）

路径	描述
asterinas_integration.md	Asterinas 接入指南（6 个步骤）
benchmark_contract.md	正式性能测试合同（环境、FIO 参数、元数据、统计口径）
cryptsetup_compatibility.md	cryptsetup 兼容方案说明（三层架构）
defense_outline.md	答辩提纲（15 分钟建议节奏、高频问题与回答）
design.md	设计说明（目标、核心接口、模块组成）
environment_status.md	开发环境状态（已迁移至 D 盘）
filebench.md	Filebench 基准说明
final_report.md	最终实验报告（8 章）
innovation.md	技术亮点与验证口径
performance.md	性能测试入口
requirement_mapping.md	赛题技术指标对应表
rootfs_protection.md	根文件系统保护方案（5 阶段）
sqlite_benchmark.md	SQLite 固定 workload 说明
technical_defense_guide.md	决赛技术答辩指南
test.md	测试方案
evidence/	正式证据根目录
evidence/README.md	证据索引
evidence/stabilization/	构建恢复和正确性证据
evidence/stabilization/build-recovery/	kernel build 日志
evidence/stabilization/correctness-kvm/	R01–R03、C01–C05、V02–V07 串口日志

路径	描述
evidence/final-performance-20260811/	最终性能证据
evidence/final-performance-20260811/PERFORMANCE_VALIDATION_REPORT.md	中文性能验证报告
evidence/final-performance-20260811/PERFORMANCE_MATRIX.csv	正式 FIO 矩阵（46 PASS + 6 N/A）
evidence/final-performance-20260811/asterinas-fio/	Asterinas KVM 原始 FIO JSON 和 CSV（13+7 项）
evidence/linux-qemu-fio-20260816/	Linux QEMU virtio FIO JSON 和 CSV（raw 13 + crypt 13 + verity 读 7）
evidence/linux-qemu-filebench-sqlite-20260817/	Linux QEMU guest Filebench / SQLite 正式对照
evidence/final-performance-20260811/filebench-valid/	Linux host Filebench 归档，不再引用
evidence/final-performance-20260811/sqlite/	Linux host SQLite 归档，不再引用
evidence/final-performance-20260811/boot-matrix/	启动 15 轮记录
evidence/stack/	组合栈启动证据
evidence/linux-xts-plain64.sha256	Linux XTS 固定向量 SHA-256
document/	技术实现文档 LaTeX 源码和 PDF，与 design_doc/ 互为补充
document/main.tex	技术文档主文件
document/chapters/	9 个章节（01_introduction 至 09_development）

路径	描述
document/appendices/	附录（术语表、命令参考）
document/figures/	技术文档插图（架构图、流程图、截图等）
document/main.pdf	编译后的技术文档 PDF（约 120 页）
design_doc/	设计赛道文档 LaTeX 源码和 PDF
design_doc/design_doc.tex	设计赛道文档主文件
design_doc/chapters/	10 个章节（01_goal 至 10_harvest）
design_doc/preamble.tex	LaTeX 导言区配置
design_doc/figures/	插图（甘特图、时间线、测试金字塔）
design_doc/design_doc.pdf	编译后的设计赛道文档 PDF（约 80 页）

9.5 关键证据文件说明

证据文件	内容说明
R01-cryptsetup-persistent-cache.log	KVM 中 cryptsetup 2.7.5 open/read/close 完整流程 PASS
R02-crypt-rootfs-persistent-cache.log	dm-crypt 加密根文件系统启动与 pivot_root PASS
R03-verity-rootfs-persistent-cache.log	dm-verity 校验根文件系统启动与 pivot_root PASS
C01-C05-kvm.log	正确/错误密钥、4/32/256 KiB 往返测试 PASS
V02-wrong-root.log	错误根哈希拒绝 mapper 创建，status=integrity-rejected
V03-targeted-probe.log	首个数据块（物理块 0）单字节篡改拒绝
V04-targeted-probe.log	中间数据块（物理块 32768）单字节篡改拒绝

证据文件	内容说明
V05-targeted-probe.log	最后一个数据块（物理块 65535）单字节篡改拒绝
V06-targeted-probe.log	叶子哈希块（物理块 65536）单字节篡改拒绝
V07-targeted-probe.log	中间哈希块（物理块 66048）单字节篡改拒绝
PERFORMANCE_MATRIX.csv	46 个 PASS 单元格 + 6 个 N/A 的完整 FIO 矩阵
PERFORMANCE_VALIDATION_REPORT.md	最终中文性能报告（含环境、口径、结论）

9.6 与大赛提交要求对照

表 9.10: 大赛提交要求对照表

提交要求	本项目对应文件	位置
项目源代码	crates/、asterinas_port/、scripts/	根目录
技术设计文档	docs/document/main.pdf	docs/document/
设计赛道文档	docs/design_doc/design_doc.pdf	docs/design_doc/
用户手册	README.md	根目录
正确性证据	12 组测试日志（R01–R03、C01–C05、V01–V07）	docs/evidence/stabilization/
性能证据	FIO、Filebench、SQLite、Boot 原始数据	docs/evidence/final-performance-20260811/
演示视频	作品演示视频.mp4	根目录
答辩 PPT	presentation_project.pptx	根目录
作品进展汇报幻灯片	作品进展汇报幻灯片.pptx	根目录
AI 工具声明	docs/ai_tool_disclosure.md	docs/

9.7 构建与测试产物（不提交）

以下文件由脚本生成，位于 target/ 目录，不纳入代码仓库：

产物	说明
target/crypt-rootfs.img 及 *.env	dm-crypt 保护根镜像（由 build_crypt_rootfs.sh 生成）
target/verity-rootfs.img 及 *.env	dm-verity 保护根镜像（由 build_verity_rootfs.sh 生成）
target/stack-rootfs.img 及 *.env	组合栈根镜像（由 build_stack_rootfs.sh 生成）
target/benchmark/	性能测试中间文件和日志
target/xts-compat/	Linux XTS 兼容性验证临时文件
target/final-demo/	最终演示临时输出
target/osdk/	Asterinas 内核构建产物（ISO）

9.8 小结

本章完整列出了提交仓库的目录结构和文件描述，涵盖了源代码（约 3000 行用户态 Rust + 约 4000 行内核补丁）、补丁（7 个）、脚本（约 30 个）、证据（约 100+ 个日志/JSON/CSV 文件）、文档（约 200 页 LaTeX）和演示材料。所有关键证据文件均有明确路径说明，便于评委复核。

项目文件组织遵循以下原则：

1. 核心代码与平台适配分离（crates/ vs asterinas_port/）；
2. 文档集中管理（docs/）且分类清晰（document、design_doc、evidence）；两份 PDF 互为补充；
3. 脚本可执行且自包含（scripts/），支持一键演示和验证；
4. 证据原始格式保存（不依赖手工修改数字），确保可复核性。

第 10 章　比赛收获

本章收获速览

收获维度	核心成长	量化成果
技术能力	Rust 系统编程、密码学工程、内核开发、性能工程	约 9000 行 Rust+2000 行脚本+200 页文档
工程能力	大型项目协作、自动化测试、文档写作	28 次 Git 提交、7 个补丁、30+ 脚本
团队协作	分工互补、共识决策、代码审查	3 人协作 50 天、12 组测试全部 PASS
认知提升	Rust OS 生态理解、安全存储认知	验证了 5 项技术探索、5 个未来方向

10.1　概述

历时近两个月的开发与测试（2026 年 6 月 23 日至 8 月 14 日，共 50 天），我们完成了从用户态核心实现到星绽内核集成、从正确性验证到性能基准测试的完整开发链路。这段经历不仅让我们在技术和工程能力上获得了显著成长，也加深了我们对 Rust 系统编程和操作系统内核开发的理解。本章从技术成长、工程能力、团队协作、生态认知和个人感悟五个维度总结本次比赛的收获。

10.2　技术成长

10.2.1　Rust 系统编程的深入实践

本项目是我们在 Rust 语言上进行的最深入的系统级编程实践，累计编写约 9000 行 Rust 代码（用户态约 3000 行，内核态约 4000 行，补丁约 2000 行）。通过实现 `BlockDevice` trait、AES-256-XTS 加密引擎（约 350 行）、Merkle 哈希树（约 150 行）等核心模块，我们深刻体会到了 Rust 的所有权系统和类型系统在系统编程中的价值：

- **编译期内存安全**：Rust 的借用检查器在编译期杜绝了悬垂指针、use-after-free 和缓冲区溢出等内核常见内存安全问题，使我们可以更专注于逻辑正确性而非内存管理。在整个开发过程中，我们没有遇到任何内存安全问题导致的崩溃——这在 C 语言内核开发中是难以想象的；
- **零成本抽象**：通过 trait 和泛型实现了高度可复用的代码，同时保持了与手写 C 代码相当的性能。AES-NI 路径下的加密吞吐达到 93+ MiB/s（256K 随机写），抽象层没有引入 measurable 的运行时开销；
- `no_std` 环境开发：在无标准库的内核环境中，我们学会了使用 `alloc`、核心库和自定义 panic handler 构建可靠的系统组件。内核组件成功运行于 Asterinas 的受限环境中，证明了 Rust 在 `no_std` 场景下的成熟度；
- **类型系统编码安全属性**：通过 `TargetCapabilities` 等类型设计，将安全策略在编译期编码到类型系统中，避免了运行时检查的开销和遗漏风险。

10.2.2 密码学工程实践

实现 AES-256-XTS 和 SHA-256 的过程让我们深入理解了密码学算法的工程化要求：

- **标准合规**：严格遵循 NIST SP 800-38E（XTS-AES）和 FIPS 180-4（SHA-256）标准，确保算法实现的正确性。通过标准测试向量验证了 SHA-256 的正确性（空字符串、`"abc"`）；
- **兼容性验证**：通过与 Linux 内核逐字节比对，验证了 `plain64` IV 生成和 XTS 模式的正确性。固定向量 SHA-256 完全匹配（`4334c17f...`），理解了“算法实现”与“协议兼容”之间的差距——两者必须完全一致才能保证互操作性；
- **硬件加速集成**：在保存/恢复 FPU/XMM 状态的前提下集成 AES-NI 指令（`ostd/aesni.rs`，约 174 行），学习了内核中处理 SIMD 寄存器的特殊约束——必须在禁用抢占和中断的情况下操作 XMM 寄存器，防止上下文切换破坏加密状态；
- **密钥管理**：实现了运行时 mapper 中密钥的显式清除（`table.key.fill(0)`），防止密钥残留在内存中被后续攻击者读取。

10.2.3 操作系统内核开发

通过 `aster-dm` 组件接入 Asterinas 内核（7 个补丁，约 4000 行内核代码），我们对操作系统内核的关键子系统有了更深入的理解：

- **块设备与 BIO 系统**：理解了块设备驱动的 I/O 路径、BIO 的提交与完成机制、同步与异步 I/O 的处理差异。在 Asterinas 中，BIO 通过 `enqueue` 异步提交，通过回调完成，与 Linux 的 `submit_bio` 机制类似但接口不同。我们成功实现了完整的 BIO 生命周期管理；
- **启动过程与根文件系统**：掌握了内核启动参数解析（`aster-cmdline` 框架）、initramfs

的作用、pivot_root 的系统调用语义。通过两级引导方案（bind mount + chroot）成功解决了 Asterinas rootfs 特殊性导致的 pivot_root 限制；

- **内核同步与并发**：在 DmDevice 中处理多线程 BIO 并发时，学会了使用 spin::Mutex（而非睡眠锁）、原子操作（AtomicU64）和禁用抢占（disable_preempt）等内核同步原语。理解了在内核态中“不能睡眠”的特殊约束；
- **设备文件系统**：实现了 /dev/mapper/control 控制节点（约 520 行），理解了字符设备与块设备的注册与管理，以及 ioctl 系统调用的内核处理路径。

10.2.4 性能工程

性能测试与优化是本次比赛的重要环节，我们在这一过程中收获颇丰：

- **基准测试方法论**：学会了设计可重复的性能测试，包括固定环境、交替轮次（每轮顺序交替消除缓存效应）、保留原始数据（JSON/CSV）和报告中位数/标准差（而非选择最好一轮）。这套方法论确保了性能数据的可信度和可复现性；
- **性能归因**：通过在内核热路径中嵌入原子计数器和阶段计时（11 个阶段计数器），学会了精确量化每个子系统的开销。正式性能数据显示 dm-crypt 中下层等待占 97.84%，AES-XTS 加密仅占 1.82%；dm-verity 中下层等待占 93.01%，数据 SHA-256 占 5.74%——这些数据指明了后续优化的真正瓶颈；
- **优化验证**：通过 A/B 对照（scratch pool on/off、hash cache on/off）验证了优化措施的实际收益，避免了“拍脑袋优化”。stats=0/1 的对照实验还量化了性能归因本身的观察者开销（<0.5%），证明计时不会显著影响 I/O 性能。

10.3 工程能力提升

10.3.1 大型项目开发与协作

项目代码量超过 11000 行（约 9000 行 Rust + 约 2000 行 Shell/Python），涉及 Rust 工作区、内核补丁、Shell/Python 脚本和 LaTeX 文档等多个技术栈。我们在以下方面获得了显著提升：

- **版本管理**：使用 Git 进行 28+ 次提交，通过补丁序列（0001–0007）管理内核集成的演进，养成了“原子化提交”的习惯——每次提交只做一件事，提交信息清晰描述“做了什么、为什么做”；
- **代码审查**：核心模块的变更经过团队成员交叉审查，学会了从正确性、安全性和可维护性多个维度审视代码。特别是安全敏感模块（加密、校验、缓存）的审查尤为严格；
- **测试驱动开发**：先编写用户态单元测试验证算法，再接入内核；先通过正确性测试，再进行性能优化。这种渐进式开发模式使我们可以快速迭代，用户态作为“安全网”捕获了大量早期问题；

- **技术决策记录**：重要决策（如补丁组织方式、性能优化优先级）形成书面记录，确保决策可追溯、可回溯。

10.3.2 自动化与可复现性

我们构建了一套完整的自动化测试与验证体系：

- 一键演示脚本（`final_demo.sh`，约 200 行）覆盖用户态测试、Linux 兼容性验证和 KVM 证据检查，最终输出 `FINAL: PASS`；
- 性能测试脚本支持自动发现和恢复（断点续跑），五轮矩阵可一键启动，运行时间约 2-3 小时；
- 静态检查脚本（`check_benchmark_package.py`、`review_release_check.py`）确保提交材料的完整性和一致性，防止遗漏关键证据；
- 所有测试保留原始证据（串口日志、JSON、CSV），不依赖手工修改数字，确保可复核性。

这些经验使我们认识到：在系统软件项目中，自动化工具链和可复现性不仅是效率工具，更是保证质量的关键基础设施——它们让“评审”变成了“复核”，让评委可以验证而非仅凭信任接受我们的结论。

10.3.3 文档写作与学术表达

本次比赛要求提交技术文档、设计赛道文档、答辩 PPT 和演示视频，这对我们的文档写作和学术表达能力是一次极好的锻炼：

- 使用 LaTeX 编写结构化的技术文档（约 200 页），学会了图表（`figure`、`table`）、代码清单、交叉引用和文献管理（`BibTeX`）的规范使用；
- 将技术细节转化为清晰的口头表达（答辩提纲、FAQ），学会了面向不同受众（评委、同行）的表述方式——技术文档追求精确和完整，答辩 PPT 追求清晰和有力；
- 制作演示视频时，学会了在有限时间（5 分钟）内呈现核心亮点的叙事技巧——从“为什么”到“怎么做”再到“做到了什么”；
- 文档与代码的同步迭代：每个功能模块完成后及时更新文档，避免了“代码改完、文档滞后”的常见问题。

10.4 团队协作收获

10.4.1 分工与互补

团队成员的技术背景各有侧重，形成了良好的互补：

- 加密与内核实现、性能优化、文档与测试三个方向均有专人负责，同时交叉支援。核心开发者专注算法和内核，测试工程师保证数据质量，文档工程师确保成果可

呈现；
- 通过每周线上会议和即时通讯，避免了“各自为政”导致的信息孤岛。平均每天 3-5 条技术消息交流，信息同步及时；
- 遇到难题时（如 `pivot_root` 限制、性能测试真实性问题），团队集体讨论、共同解决。没有人“自己扛”，也没有人“事不关己”。

10.4.2 共识决策与代码审查

- 重要技术决策（BIO 生命周期设计、缓存安全策略、性能优化优先级）在会议中集体讨论后实施，形成共识文档。决策质量更高，风险更可控；
- 核心模块的代码变更须至少一名其他成员审查后方可合并。补丁提交前在本地进行完整的 `cargo test` 和 KVM 正确性验证；
- “代码改、文档跟”的习惯使文档与代码始终保持一致，也帮助我们在撰写文档的过程中发现了设计中不清晰或需要改进的地方。

10.4.3 团队成员感想

- **潘利源**：“从用户态 `demo-crypt` 到内核中 `pivot_root` 切换成功，每一步都充满挑战。最难忘的是调试 BIO 生命周期问题的夜晚——当串口终于输出 `PASS system root switched to protected mapper` 时，那种成就感难以言表。这次比赛让我真正理解了‘内存安全’在操作系统层面的含义，也让我对 Rust 的未来充满信心。特别是当意识到同样的 bug 在 C 语言中可能是一个难以复现的 UAF（use-after-free）漏洞，而在 Rust 中它只是一个生命周期编译错误时，我对 Rust 的价值有了更深的认识。”
- **陈韵**：“性能测试的真实性问题给我上了重要一课：数据漂亮不等于数据正确。早期假 mapper 测试产生的高吞吐量曾让我们兴奋，但冷静分析后才发现是测试环境的问题。重新设计验证流程、修复 runner 的过程让我明白：在工程中，‘诚实’比‘好看’更重要。这次经历也让我学会了如何设计可审计的测试框架——每个数据点都能追溯到原始证据，而不是只看最终数字。”
- **张乐桁**：“作为负责文档和 PPT 的成员，我最初担心自己‘离代码太远’。但当我把散落在各处的技术细节整理成结构化的文档、把复杂的性能数据转化为清晰的图表时，我意识到文档不是代码的附属品，而是项目的重要组成部分。好的文档能让代码的价值被更多人看到。此外，使用 LaTeX 排版 200 页技术文档的经历也极大提升了我的工具链能力——从‘会用 Word’到‘能驾驭 LaTeX’是一个质的飞跃。”

10.5 对 Rust OS 生态的认识

通过本次比赛，我们对 Rust 操作系统生态的现状和发展方向有了更深入的认识：

1. **优势已得到验证**：Rust 的所有权和类型系统在系统编程中提供了无与伦比的内存安全保证——整个项目中未出现任何内存安全漏洞导致的崩溃，这在 C 语言内核开发中是几乎不可能做到的。Asterinas 的 Framekernel 架构将大部分内核组件运行在用户态，进一步缩小了 TCB，为可信计算提供了良好的基础。本项目验证了在 Rust OS 中实现与 Linux 兼容的可信存储是完全可行的；
2. **挑战依然存在**：Rust 内核生态仍处于早期阶段。调试工具（如 GDB 对 Rust 调试符号的支持）与 C 内核相比仍有差距；性能分析工具（如 perf 对 Rust 符号的解析）需要额外配置；`no_std` 环境下的开发需要额外注意依赖选择（避免使用标准库功能）和内存分配策略（谨慎使用 `alloc`）；
3. **未来探索方向明确**：Rust 在可信存储、机密计算和形式化验证方面具有天然优势。本项目验证了认证后缓存、BIO 复用和硬件加速等技术在 Rust OS 中的可行性，为后续的可信启动、远程证明和密钥管理奠定了基础。随着 Rust 内核生态的成熟，我们可以期待更多 Rust 原生的安全特性被引入操作系统开发。

10.6 关键技术指标达成总结

表 10.1: 项目成果与赛题指标对照

赛题指标	**达成状态**	**关键证据**
dm-verity 基础功能	完全达成	R03/V01 PASS，V03–V07 篡改拒绝
dm-crypt 基础功能	完全达成	R01/C01–C05 PASS，XTS 逐字节兼容
根文件系统加密	完全达成	R02 PASS，`pivot_root` 成功
性能与启动要求	完全达成	46 PASS 单元格，启动开销 <3%
技术探索创新	完全达成	AES-NI、hash cache、scratch pool、BIO 复用

10.7 未来展望详细路线图

方向	具体目标	预期收益
完整 LUKS2 支持	PBKDF、密钥槽、header 管理	与标准 cryptsetup 完全兼容
异步 BIO 优化	解密/校验移至专用 worker	减少 IRQ 路径延迟，提升吞吐
可信启动集成	与 TEE/远程证明结合	建立完整信任链
更多 cipher 支持	AES-CBC-ESSIV 等	扩大 Linux 兼容性
上游贡献	代码回馈 Asterinas 社区	推动 Rust OS 生态发展

10.8 个人感悟

10.8.1 潘利源

“从用户态 `demo-crypt` 到内核中 `pivot_root` 切换成功，每一步都充满挑战。最难忘的是调试 BIO 生命周期问题的夜晚——当串口终于输出 `PASS system root switched to protected mapper` 时，那种成就感难以言表。这次比赛让我真正理解了‘内存安全’在操作系统层面的含义，也让我对 Rust 的未来充满信心。特别是当意识到同样的 bug 在 C 语言中可能是一个难以复现的 UAF 漏洞，而在 Rust 中它只是一个生命周期编译错误时，我对 Rust 的价值有了更深的认识。”

10.8.2 陈韵

“性能测试的真实性问题给我上了重要一课：数据漂亮不等于数据正确。早期假 mapper 测试产生的高吞吐量曾让我们兴奋，但冷静分析后才发现是测试环境的问题。重新设计验证流程、修复 runner 的过程让我明白：在工程中，‘诚实’比‘好看’更重要。这次经历也让我学会了如何设计可审计的测试框架——每个数据点都能追溯到原始证据，而不是只看最终数字。”

10.8.3 张乐桁

“作为负责文档和 PPT 的成员，我最初担心自己‘离代码太远’。但当我把散落在各处的技术细节整理成结构化的文档、把复杂的性能数据转化为清晰的图表时，我意识到文档不是代码的附属品，而是项目的重要组成部分。好的文档能让代码的价值

被更多人看到。此外，使用 LaTeX 排版 200 页技术文档的经历也极大提升了我的工具链能力——从‘会用 Word’到‘能驾驭 LaTeX’是一个质的飞跃。”

10.9 感谢

感谢全国大学生计算机系统能力大赛组委会提供这一宝贵的实践平台，让我们有机会将课堂所学应用到真实的操作系统开发中。感谢星绽操作系统开源社区提供的优秀框架和开发文档——特别是 OSTD 框架和 `aster-block` 组件，为我们的内核集成提供了清晰的接口。感谢指导老师在关键节点的点拨和支持。

最重要的是，感谢团队每一位成员的付出与信任。从最初的架构讨论到最终的文档交付，我们共同经历了代码 panic 的焦虑、性能调优的煎熬和终版提交的喜悦。这段经历将成为我们大学生活中最珍贵的回忆之一。

10.10 小结

本次比赛是一次难得的全方位成长机会。我们在以下五个维度都获得了显著提升：

1. **技术能力**：Rust 系统编程（约 9000 行）、密码学工程（AES-XTS + SHA-256）、内核开发（7 个补丁，4000 行）、性能工程（五轮基准，46 个单元格）；
2. **工程能力**：大型项目协作（28 次 Git 提交）、自动化测试（30+ 脚本）、文档写作（200 页 LaTeX）；
3. **团队协作**：3 人协作 50 天、分工互补、共识决策、代码审查；
4. **认知提升**：对 Rust OS 生态有了系统认识，理解了 Rust 在系统编程中的价值与局限；
5. **个人成长**：每位成员都在技术自信和工程视野上有了质的提升。

更重要的是，我们验证了 Rust 在操作系统安全存储领域的潜力——用内存安全的语言构建与 Linux 兼容的可信存储是完全可行的。这段经历将激励我们在系统软件的道路上继续前行。